FINAL PROJECT

Helena Rolle

July 21st, 2025

```
list.files(pattern = "\\.log$")
```

```
## character(0)
```

INTRODUCTION

This project will broadly scoped on two main fronts: *AnalysisofRaw* data and the *AnalysisofNormalized* data.

The focus of this project is to make causual inferences about the relationship among grain yield and seeding rate use agricultural data from 2017 to 2020. Some of the questions to be answered by the project: How is fertility determined? Does yield and/or seeding rate of previous year(s) determine yield of current year(s)? or Is there a relationship between the yield of a given year and seeding/yield of prior year(s)?

To analyze, individual yield and seeding points (data files with yield and seeding information with GPS position) need to be combined.

SPATIAL CONTEXT: We will start with six files, and aggregate the data by first creating a set of grid cells, then merging the framed cells into a single data set. There is the need to combine individual yield and seeding maps (data files with yield and seeding information tagged with a GPS position). See the table below for the data files to be aggregated.

MODELING OBJECTIVES and GOALS: To explore and model the relationships between yield production and potential predictors such as previous year's yield, seeding rate, and spatial location, using aggregated and normalized spatial data. The ultimate goal is to predict a current-year's yield by examining the causal structure relative to other variables - crop rotation influence, applied rate, etc.

File Name	Crop	Year	Original Variable	Aggregated Variable
`2017 Soybeans Harvest.csv`	Soybean	2017	`Yield`	`Y17`
`2018 Corn Seeding.csv`	Corn	2018	`AppliedRate`	`AR18`
`2018 Corn Harvest.csv`	Corn	2018	`Yield`	`Y18`
`2019 Soybeans Harvest.csv`	Soybean	2019	`Yield`	`Y19`
`2020 Corn Seeding.csv`	Corn	2020	`AppliedRate`	`AR20`
`2020 Corn Harvest.csv`	Corn	2020	`Yield`	`Y20`

1. RAW DATA ANALYSIS

A. DATA

Logged in six different files, the data can be viewed as two types: the seed application and production yield. We note that the initial data starts with the 2017 yield without any 2016 prior datasets (seeding or applied rate). Also, there is no 2019 seeding rates to address either the 2019 or 2020 yield.

The seeding files, capture data such as: plotted geo-location, the time of the planting, applied, and control rate of the seeding. The yield (harvest) files, captured similar data items including wet mass,and moisture.

Common fields in each file type include: Long, Lat, time, and distance. For analysis, we note that these common fields would allow for the datasets to be merged. Further a mathematical perspective of *Lat* and *Lon* with give rise to a *Row* and *Column* variable from which a *Cell* variable can be mapped.

In terms of data cleaning, checks are made for missing data (NULL, NAs, etc), and numerical values since instructions call for calculation of the mean, observation counts and graph plots.

Intermediatary steps will see data manipulation of the form where a dataframe may be converted from long form to wide form or vice versa. Also some maniuplations/outputs may require specific variables: either Lat/Lon or Row/Col or Cell mappings.

Aggregate Raw data

Each combination of Row and Column will uniquely identify grid cells. These will be used to aggregate the data.

Called Aggregation Function

```
aggregate_by_grid <- function(data, value_col, cell_size = 50) {
  # Compute variables: NRow, Column, Cell
  data$Row <- ceiling(data$Latitude / cell_size)
  data$Column <- ceiling(data$Longitude / cell_size)
  data$Cell <- data$Row * 1000 + data$Column

  # Subset to keep only needed columns
  df_sub <- data.frame(Cell = data$Cell,
                       Row = data$Row,
                       Column = data$Column,
                       Latitude = data$Latitude,
                       Longitude = data$Longitude,
                       Value = data[[value_col]])

  # Compute mean per grid cell - requirement
  agg_mean <- aggregate(Value ~ Cell + Row + Column, data = df_sub, FUN = mean, na.rm = TRUE)
  colnames(agg_mean)[4] <- "Mean"

  # Compute count per grid cell - requirement
  agg_count <- aggregate(Value ~ Cell + Row + Column, data = df_sub, FUN = length)
  colnames(agg_count)[4] <- "Count"

  # Merge results of mean and count values
```

```
  agg_result <- merge(agg_mean, agg_count, by = c("Cell", "Row", "Column"))

  return(agg_result)
}
```

1. Data Input - Read Files

```
# Load data and drop unnamed first column from files
soyHarvest_2017 <- read.csv("Data/2017 Soybeans Harvest.csv")[, -1]
cornSeed_2018 <- read.csv("Data/2018 Corn Seeding.csv")[, -1]
cornHarvest_2018 <- read.csv("Data/2018 Corn Harvest.csv")[, -1]
soyHarvest_2019 <- read.csv("Data/2019 Soybeans Harvest.csv")[ ,-1]
cornSeed_2020 <- read.csv("Data/2020 Corn Seeding.csv")[, -1]
cornHarvest_2020 <- read.csv("Data/2020 Corn Harvest.csv")[, -1]

# Aggregate by Yield/Applied Rate
agg_grid_y17 <- aggregate_by_grid(soyHarvest_2017, "Yield")
colnames(agg_grid_y17)[4] <- "Y17"
#head(agg_grid_y17) - as verification

agg_grid_ar18 <- aggregate_by_grid(cornSeed_2018, "AppliedRate")
colnames(agg_grid_ar18)[4] <- "AR18"

agg_grid_y18  <- aggregate_by_grid(cornHarvest_2018,  "Yield")
colnames(agg_grid_y18)[4] <- "Y18"

agg_grid_y19  <- aggregate_by_grid(soyHarvest_2019,   "Yield")
colnames(agg_grid_y19)[4] <- "Y19"

agg_grid_ar20 <- aggregate_by_grid(cornSeed_2020,     "AppliedRate")
colnames(agg_grid_ar20)[4] <- "AR20"

agg_grid_y20  <- aggregate_by_grid(cornHarvest_2020,  "Yield")
colnames(agg_grid_y20)[4] <- "Y20"
```

2. Data Selection

```
# only file rows where the count >=30: done before data merge
cntVal <- 30
agg_grid_y17  <- agg_grid_y17[agg_grid_y17$Count >= cntVal, ]
agg_grid_ar18 <- agg_grid_ar18[agg_grid_ar18$Count >= cntVal, ]
agg_grid_y18  <- agg_grid_y18[agg_grid_y18$Count >= cntVal, ]
agg_grid_y19  <- agg_grid_y19[agg_grid_y19$Count >= cntVal, ]
agg_grid_ar20 <- agg_grid_ar20[agg_grid_ar20$Count >= cntVal, ]
agg_grid_y20  <- agg_grid_y20[agg_grid_y20$Count >= cntVal, ]
```

3. Data Merge

** Merging of all 6 data files **

```r
# Merge all filtered datasets into a single one
Combined.dat <- Reduce(function(x, y) merge(x, y, by = c("Cell", "Row", "Column"), all = FALSE),
                       list(agg_grid_y17, agg_grid_ar18, agg_grid_y18,
                            agg_grid_y19, agg_grid_ar20, agg_grid_y20))
```

```
## Warning in merge.data.frame(x, y, by = c("Cell", "Row", "Column"), all =
## FALSE): column names 'Count.x', 'Count.y' are duplicated in the result
## Warning in merge.data.frame(x, y, by = c("Cell", "Row", "Column"), all =
## FALSE): column names 'Count.x', 'Count.y' are duplicated in the result
```

```
## Warning in merge.data.frame(x, y, by = c("Cell", "Row", "Column"), all =
## FALSE): column names 'Count.x', 'Count.y', 'Count.x', 'Count.y' are duplicated
## in the result
```

```r
# Quick preview - merged data
head(Combined.dat)
```

```
##    Cell Row Column      Y17 Count.x     AR18 Count.y      Y18 Count.x      Y19
## 1 10001  10      1 53.36685      96 26092.36      41 211.7009     124 39.14806
## 2 10002  10      2 52.58359     102 26339.74      42 228.9304     107 32.78266
## 3 10003  10      3 54.19551      93 26447.59      41 232.2385      83 37.58834
## 4 10004  10      4 50.76123     102 25925.29      42 237.7755     121 48.18398
## 5 10005  10      5 52.61351      83 25973.53      40 231.6592     107 51.50722
## 6 10006  10      6 54.95515     111 26260.49      40 238.1637     108 49.17503
##   Count.y     AR20 Count.x       Y20 Count.y
## 1      89 26885.62      43 165.10126     107
## 2     107 26549.57      45  96.65559     223
## 3      78 26845.56      42 179.71318     110
## 4     110 27982.25      49 197.49239     119
## 5      77 28824.57      42 192.00151     118
## 6     108 28857.66      43 191.52483     112
```

4. Data file Cleanup

```r
# Remove any column that starts with "Count"
Combined.dat <- Combined.dat[, !grepl("^Count", names(Combined.dat))]

# View the result
#head(Combined.dat)
nrow(Combined.dat)
```

```
## [1] 202
```

B. FORMULA

a. Raw Data Other than the basic calculation for the conversion between Longitude and Latitude to Row, Column, and Cell values, no other formula willbe used on the Raw data values. A general generic calculation of the form - `df$Row = ceiling(df$Latitude/50)`; similarly for Column conversion. Spatial data points can be unravelled as: `df$Cell = df$Row * 1000 + df$Col` where #rows <=1000.

C. ALGORITHM - Project Approach

This is based on two paths:*Raw Dataset* analysis and a *Normalized Dataset* analysis.

The following provides the approach taken for each path to complete the project. The methodology will follow a simple but statistically sound approach where data is processed before being aggregated.

- This approach matches the focus: iterate within year (per sample)
- Leads to better statistical consistency
- Keeps modeling and interpretation clean

D. CODE

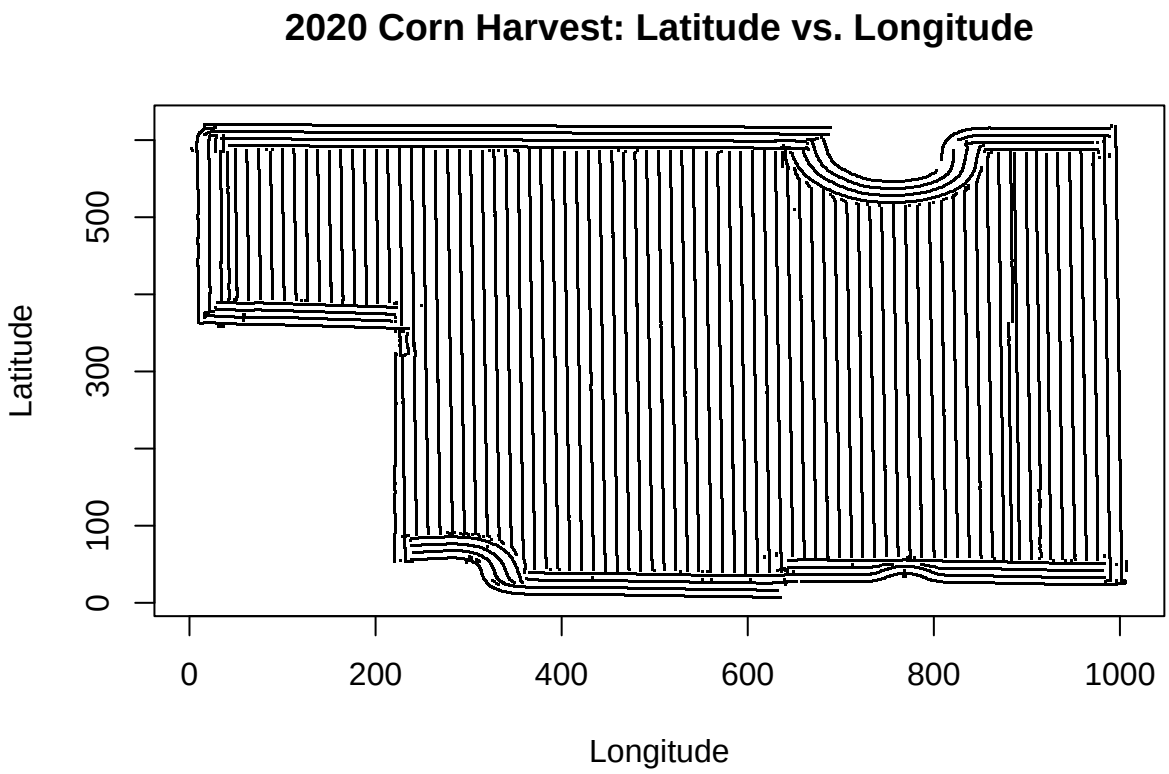
For both analyses (Raw and Normalized), the programming language of choice is R. Data will be read, cleaned and combined. The plan is to modularize the code as much as possible for understandability. Additionally R's base functions will be used. Preference will be given to simplicity over conciseness. Data integrity checks will be made to satisfy whether the incoming data is numeric, complete, or valid (no NAs, Nulls). During development print statements will be used as a means of verifying the correctness of varibles - structure, or contents but removed from the final presentation.

E. OUTPUT

For both the RAW DATA SET and NORMALIZED DATASET, the output will be progressive: Raw plots generation will be presented before Normalized ones, plots will be produced in increasing order of years, if requiores, and otherwise, outputs will be based on complexity (simple grid plot, pair plot, acyclic plot). Specifically, outputs will be those requested as listed in the project requirement. Both visual and statistical concept outputs will be generated where needed to provide answers sought by the project. If warranted, following an output, written analyses will be logged. While most of plots were done for the 2020 Yield datset, any other year's variable could have been selected, but protocol iset based on the requirement of the project. Further, the plotly graphic library will be the one of choice.

1. Simple Plot: Raw(Unnormalized) 2020 Harvest Data

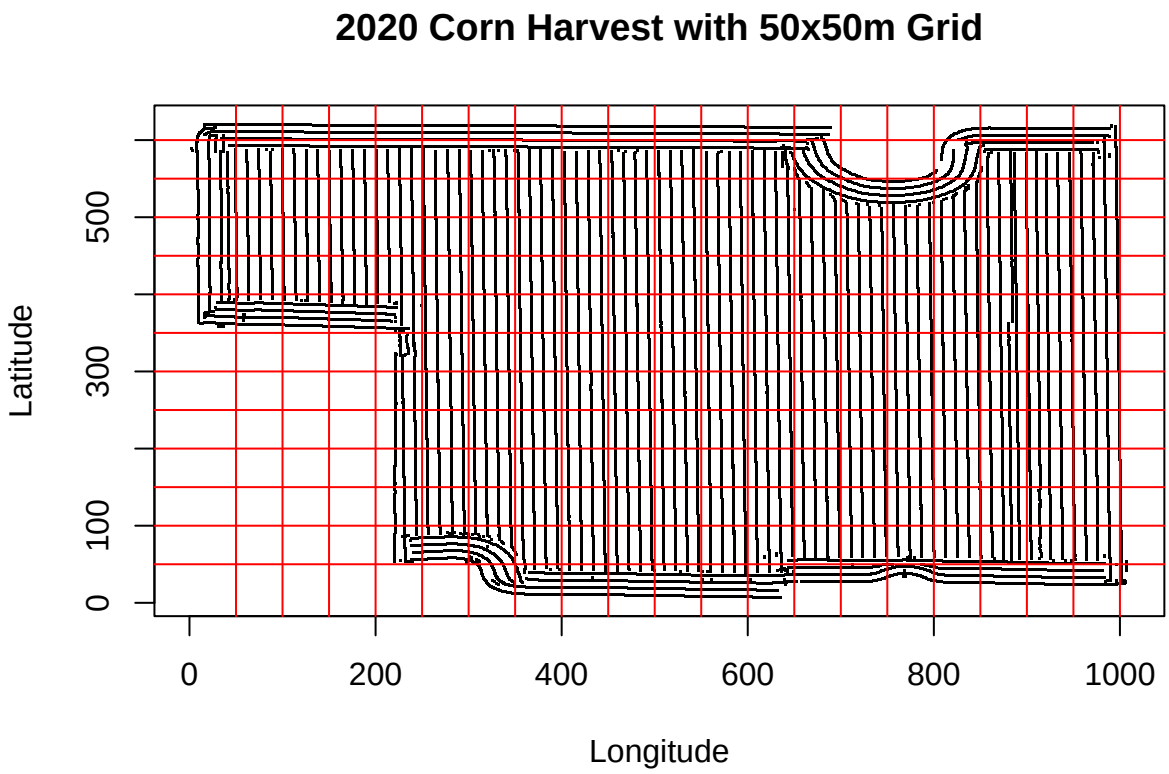
```
plot(Latitude ~ Longitude, data = cornHarvest_2020,
     pch = ".", col = "black",
     main = "2020 Corn Harvest: Latitude vs. Longitude")
```



2. 2020 Harvest Spatial Grid Format (50m x 50m)

$dim = 50mx50m$. $RowascalLat$, $ColascalLong$ and $a \cdot \text{Lat} + b \cdot \text{Long}$ (cell a linear combination of Lat and Long for cornHarvest 2020); Added abline included.

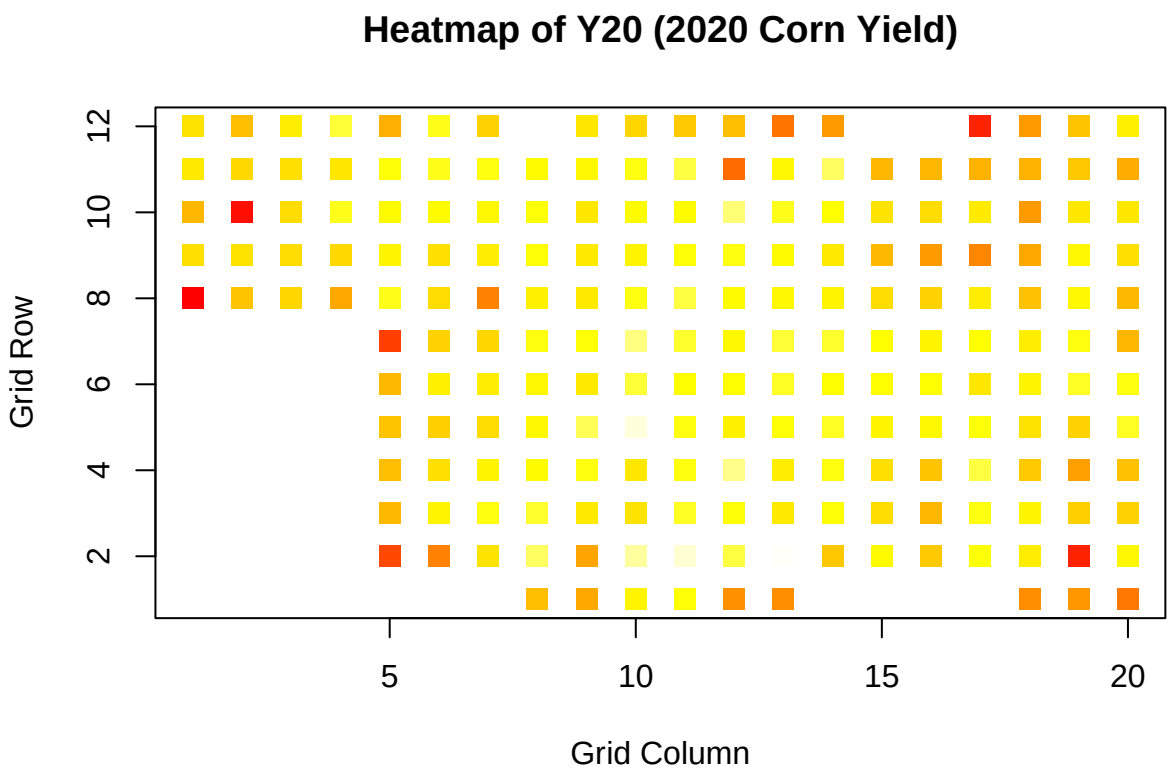
```
# Assign grid positions
cornHarvest_2020$Row    <- ceiling(cornHarvest_2020$Latitude / 50)
cornHarvest_2020$Column <- ceiling(cornHarvest_2020$Longitude / 50)
cornHarvest_2020$Cell   <- cornHarvest_2020$Row * 1000 + cornHarvest_2020$Column

plot(Latitude ~ Longitude, data = cornHarvest_2020,
     pch = ".", col = "black",
     main = "2020 Corn Harvest with 50x50m Grid",
     xlab = "Longitude", ylab = "Latitude")

# Add 50m-spaced grid lines
abline(h = 1:12 * 50, v = 1:20 * 50, col = 'red')
```



Note: This plot demonstrates the merging of data points (Longitude, Latitude) to a single point (Cell) via use of mathematical mapping. Along the journey, we note that there might also be times to unravel the almagated 'cell' to its original dimensions - *Lon* and *Lat*. Further aggregation will later be done by computing the mean and number of observation of each grid cell.

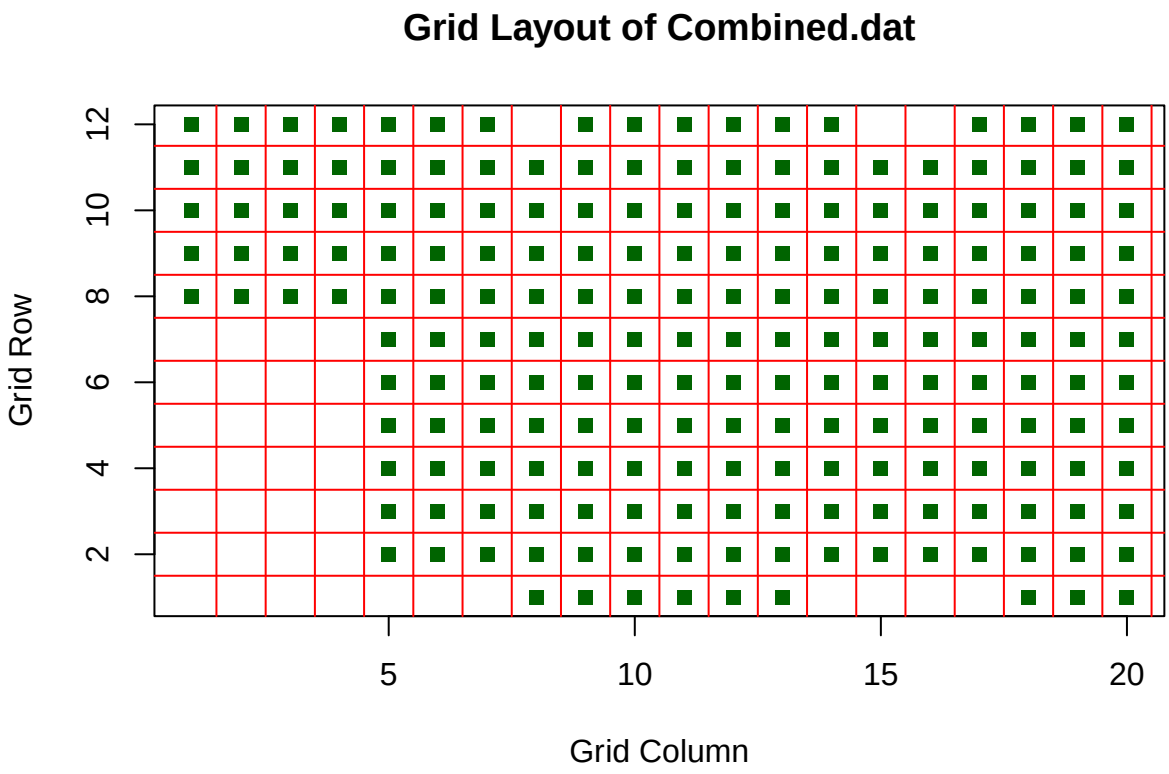
3. HeatMap: 2020 Corn Yield

```
# Filter out rows where Y20 is NA
plot_data <- Combined.dat[!is.na(Combined.dat$Y20), ]

# Create color palette
num_colors <- 100
color_bins <- cut(plot_data$Y20, breaks = num_colors)
colors <- heat.colors(num_colors)[as.numeric(color_bins)]

# Plot colored grid cells
plot(plot_data$Column, plot_data$Row,
     col = colors, pch = 15, cex = 1.5,
     xlab = "Grid Column", ylab = "Grid Row",
     main = "Heatmap of Y20 (2020 Corn Yield)")
```



4. Combined Grid Layout Plot (Raw Data)

```
# Visualize cell layout for all merged grid cells
plot(Combined.dat$Column, Combined.dat$Row,
     pch = 15, col = "darkgreen",
     xlab = "Grid Column", ylab = "Grid Row",
     main = "Grid Layout of Combined.dat")
abline(h = 1:12 + 0.5, v = 1:20 + 0.5, col = 'red')  # grid lines
```



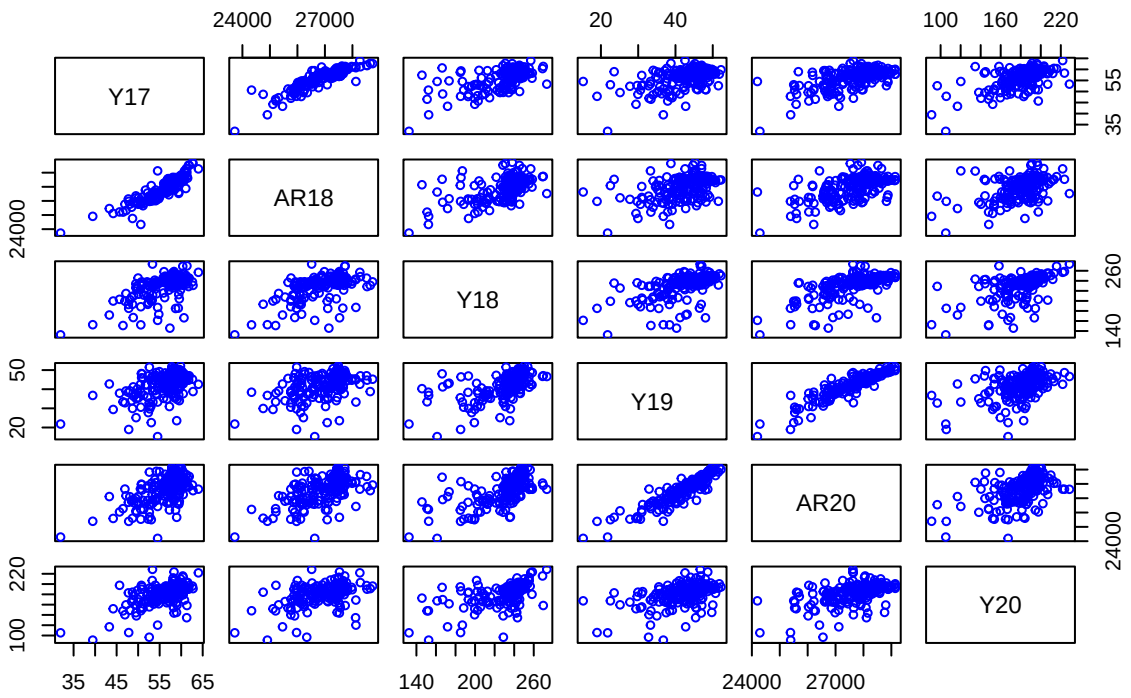

```
#head(Combined.dat)
```

Notes: Further aggregation in the form of the number of combined cells (with counts <= 30) is being used to produce this merged plot.

5. Visualized Pairs Plot

Use a pairs plot to show the relationship among the data columns for the combined data set.

```
# Check variables exist and are complete enough
pairs_vars <- c("Y17", "AR18", "Y18", "Y19", "AR20", "Y20")

# Optional: remove rows with any NA values in those columns
Combined_clean <- Combined.dat[complete.cases(Combined.dat[, pairs_vars]), ]

# Create the pairs plot
pairs(Combined_clean[, pairs_vars],
      main = "Pairs Plot: Raw Aggregated Data",
      pch = 21, col = "blue", cex.labels = 1.1,
      cex = 0.8)
```



Note: Raw Data PAIR PLOT Noting that the Pairs Plot is a symmetric diagram, we can use the lower triangular portion to access the relations between variables given that the relation between a and b is equivalent to the relation between b and a. We hereby note that while a straight line can be fitted to most of the ploted cases, only in two instances are there evidence of strong linear correlations: between AR18 and Y17 *and* between AR20 and AR19.

```
nrow(Combined_clean)
```

```
## [1] 202
```

Prepare Combined RAW Dataset - Bayesian Modeling

We filter the dataset to keep only complete cases (no missing values) across the six variables needed for the DAG models.

```
key_vars <- c("Y17", "AR18", "Y18", "Y19", "AR20", "Y20")

# Filter to complete cases
Combined.dat <- Combined.dat[complete.cases(Combined.dat[, key_vars]), ]

# Pre-checks to ensure formatting.  Define the key variable names
key_vars <- c("Y17", "AR18", "Y18", "Y19", "AR20", "Y20")

# Want data to be framed: dataframe format
if (!is.data.frame(Combined.dat)) {
```

```
  stop("Error: 'Combined2' is not a data.frame")
}

# Check all key columns are present
missing_vars <- setdiff(key_vars, names(Combined.dat))
if (length(missing_vars) > 0) {
  stop("Error: Missing columns: ", paste(missing_vars, collapse = ", "))
}

# Check each key column is numeric
for (v in key_vars) {
  if (!is.numeric(Combined.dat[[v]])) {
    stop("Error: Column '", v, "' is not numeric")
  }
}

# Remove any rows with NAs in their columns
Combined2 <- Combined.dat[complete.cases(Combined.dat[, key_vars]), ]
```

```
#head(Combined2)
nrow(Combined2)
```

```
## [1] 202
```

```
#nrow(Combined.dat)
```

6. Causal Analysis - DAG for RAW Dataset

i. DAG model for 2017 - 2018

```
library(bnlearn)
```

```
## Warning: package 'bnlearn' was built under R version 4.5.1
```

```
library(Rgraphviz)
```

```
## Loading required package: graph
```

```
## Loading required package: BiocGenerics
```

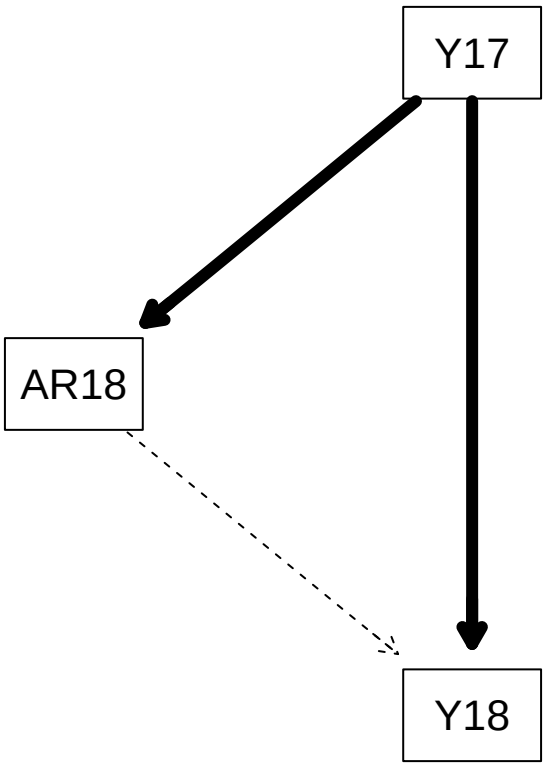
```
## Loading required package: generics
```

```
## Warning: package 'generics' was built under R version 4.5.1
```

```
##
## Attaching package: 'generics'
```

```
## The following objects are masked from 'package:base':
##
##     as.difftime, as.factor, as.ordered, intersect, is.element, setdiff,
##     setequal, union

##
## Attaching package: 'BiocGenerics'

## The following object is masked from 'package:bnlearn':
##
##     score

## The following objects are masked from 'package:stats':
##
##     IQR, mad, sd, var, xtabs

## The following objects are masked from 'package:base':
##
##     anyDuplicated, aperm, append, as.data.frame, basename, cbind,
##     colnames, dirname, do.call, duplicated, eval, evalq, Filter, Find,
##     get, grep, grepl, is.unsorted, lapply, Map, mapply, match, mget,
##     order, paste, pmax, pmax.int, pmin, pmin.int, Position, rank,
##     rbind, Reduce, rownames, sapply, saveRDS, table, tapply, unique,
##     unsplit, which.max, which.min

##
## Attaching package: 'graph'

## The following objects are masked from 'package:bnlearn':
##
##     degree, nodes, nodes<-

## Loading required package: grid
```

```r
modela.dag <- model2network("[Y17][AR18|Y17][Y18|AR18:Y17]")
fit1 = bn.fit(modela.dag, Combined2[,c('Y17','AR18','Y18')])

# fit1
strengtha <- arc.strength(modela.dag, Combined2[,c('Y17','AR18','Y18')])
strength.plot(modela.dag, strengtha)
```

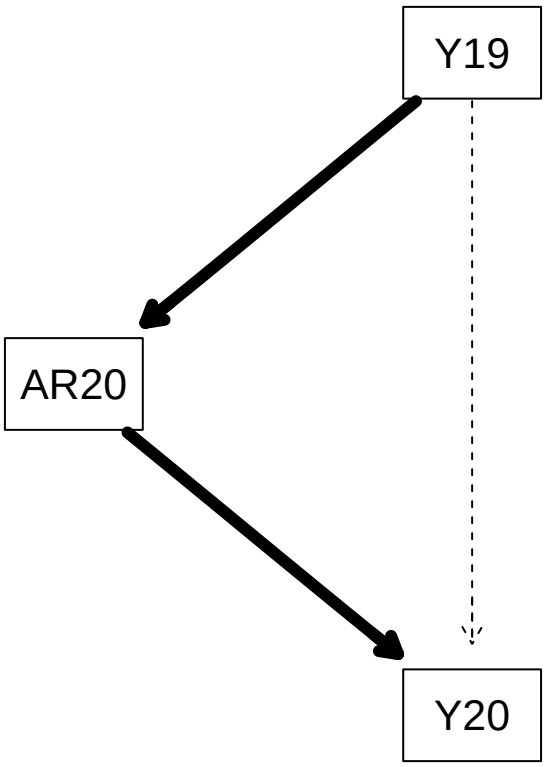


Interpretation of DAG model: 2017 - 2018

The yield of 2017 is a strong predictor for both the 2018 Applied Rate and 2017 yield but the applied rate of 2018 (AP18) is not a strong support for the same year's yield (Y18). Reference to the latter point, needs further investigation. Arising questions can be: was there measurement error, insufficient data, or collinearity (overlapping or redundant information between Y17 and AR18)?

ii. DAG model for 2019 - 2020

```
library(bnlearn)
library(Rgraphviz)

modelb.dag <- model2network("[Y19][AR20|Y19][Y20|AR20:Y19]")
fit2 = bn.fit(modelb.dag, Combined2[,c('Y19','AR20','Y20')])
#fit2
strengthb <- arc.strength(modelb.dag, Combined2[,c('Y19','AR20','Y20')])
strength.plot(modelb.dag, strengthb)
```



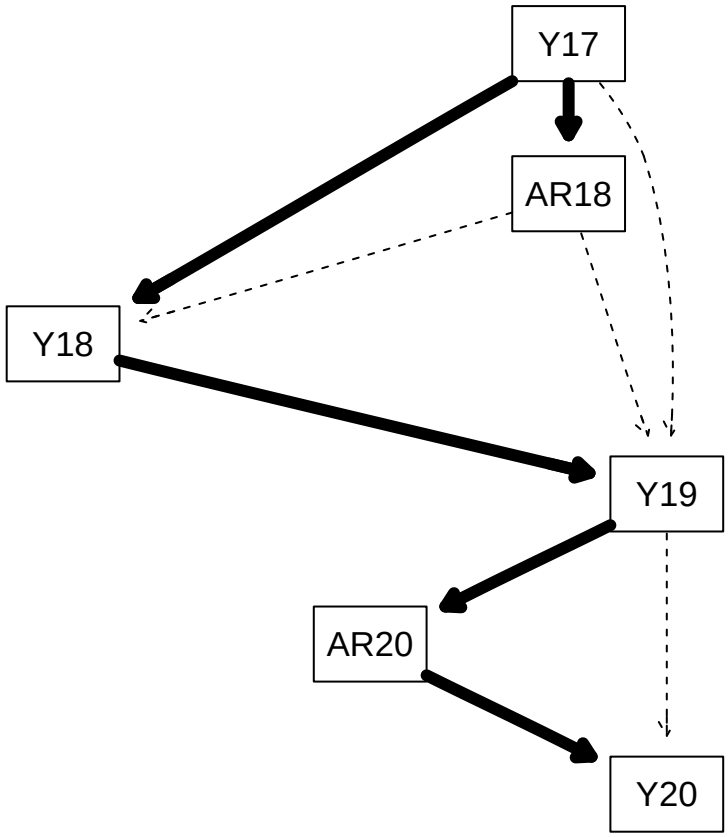
Interpretation of the DAG model 2019 - 2020

The yield of 2019 is a strong predictor for both the Applied Rate of 2020 which itself has a strong influence on the 2020 yield. That is, the yield of 2019 has a indirect consequence on the 2020 yield. We can therefore conclude that current seeding rate (AR20) and current-year factors (Y20) are tightly linked—likely due to mutual determination or shared causality under the current circumstances.

iii. DAG model for ALL Years: 2017 - 2020

```
library(bnlearn)
library(Rgraphviz)

modela.dag <- model2network("[Y17][AR18|Y17][Y18|AR18:Y17]")
fita = bn.fit(modela.dag, Combined2[,c('Y17','AR18','Y18')])

strengtha <- arc.strength(modela.dag, Combined2[,c('Y17','AR18','Y18')])
#strength.plot(modela.dag, strengtha)

modelb.dag <- model2network("[Y19][AR20|Y19][Y20|AR20:Y19]")
fitv = bn.fit(modelb.dag, Combined2[,c('Y19','AR20','Y20')])

strengthb <- arc.strength(modelb.dag, Combined2[,c('Y19','AR20','Y20')])
#strength.plot(modelb.dag, strengthb)
```

```
modelc.dag <- model2network("[Y17][AR18|Y17][Y18|AR18:Y17][Y19|Y17:AR18:Y18][AR20|Y19][Y20|AR20:Y19]")
fitc = bn.fit(modelc.dag, Combined2[,c('Y17','AR18','Y18','Y19','AR20','Y20')])

strengthc <- arc.strength(modelc.dag, Combined2[,c('Y17','AR18','Y18','Y19','AR20','Y20')])
strength.plot(modelc.dag, strengthc)
```



Interpretation of the 2017-2020 DAG model: **Notes:** - In short the effect of Y17 on Y20 follows this path: $Y17 \rightarrow Y18 \rightarrow Y19 \rightarrow AR20 \rightarrow Y20$.

- We note though that has an indirect effect on Y20 and begs a question that Y19 impacts Y20 via some intervening management decision, not necessarily a direct soil or environmental one.
- Also the yield of 2017 has a strong bearing also on the applied rate for 2018. Here again, Y17 is prominent.
- Further, we note that the graph is acyclic which is a requirement of Bayesian Models.

7. Calculated Strengths of DAG: 2017-2020 RAW Data

```
library(bnlearn)

# Learn the network structure
```

```r
modelc.dag <- hc(Combined2)  # Can be replaced by other tests:gs(), tabu()

# Compute arc strength values using test-based criterion
strength_df <- arc.strength(modelc.dag, data = Combined2, criterion = "cor")

# View the first few rows
head(strength_df)
```

```
##   from   to     strength
## 1 Cell  Row 0.000000e+00
## 2  Y19 AR20 4.345973e-84
## 3  Y17 AR18 6.864785e-75
## 4 AR20  Y18 5.787313e-27
## 5  Y18  Y17 3.450074e-08
## 6  Y17  Y20 2.184471e-07
```

Note: While similar analysis can be done for previously generated plot, we focus here on the all inclusive plot for the 4 year period. The arc from `Y19 → AR20` ; (p 4.3 ×10^{-84}) exhibits the strongest statistical connection among all variables, suggesting a highly reliable directional dependency. Similarly, the link {Y17} → AR18 ($p \approx 6.9 \times 10^{-75}$) is backed by extremely strong evidence. Though all connections are statistically significant ($p < 0.001$), the strength progressively weakens down the list, with the Y17 → Y20 link being the least strong yet notable with (p 2.2e-07).

```r
# Sort the strengths
sorted_strengths <- strength_df[order(-strength_df$strength), ]

sorted_strengths
```

```
##      from     to     strength
## 15 Column   Cell 8.838431e-03
## 14    Y19    Y20 7.530096e-03
## 13    Y20 Column 6.799116e-03
## 12    Y20    Row 5.928840e-03
## 11   AR18    Y20 3.900722e-03
## 8     Y18    Y20 3.370811e-04
## 10    Y17    Row 4.871457e-05
## 9     Y17 Column 2.803416e-05
## 7    AR20    Y17 7.030443e-07
## 6     Y17    Y20 2.184471e-07
## 5     Y18    Y17 3.450074e-08
## 4    AR20    Y18 5.787313e-27
## 3     Y17   AR18 6.864785e-75
## 2     Y19   AR20 4.345973e-84
## 1    Cell    Row 0.000000e+00
```

8. Visual Merged Raw Dataset

```r
head(Combined2)
```

```
##    Cell Row Column      Y17     AR18      Y18      Y19     AR20       Y20
```

```
## 1 10001  10      1 53.36685 26092.36 211.7009 39.14806 26885.62 165.10126
## 2 10002  10      2 52.58359 26339.74 228.9304 32.78266 26549.57  96.65559
## 3 10003  10      3 54.19551 26447.59 232.2385 37.58834 26845.56 179.71318
## 4 10004  10      4 50.76123 25925.29 237.7755 48.18398 27982.25 197.49239
## 5 10005  10      5 52.61351 25973.53 231.6592 51.50722 28824.57 192.00151
## 6 10006  10      6 54.95515 26260.49 238.1637 49.17503 28857.66 191.52483
```

```
#headc(Combined2)
nrow(Combined.dat)
```

```
## [1] 202
```

G. PRE-NORMALIZATION JUSTIFICATION

To set the stage for normalization, we will show that the scaled raw dataset, *scalComb3*, contains an inordinate amount of outliers which persists even when scaled. Outliers are data points falling outside the +/- 3 standard deviation range.

1. Outlier: Raw Combined Dataset

```
vars <- c("Y17", "AR18", "Y18", "Y19", "AR20", "Y20")

# Drop rows with missing values in selected columns
Combined3 <- Combined2[complete.cases(Combined2[, vars]), vars]

# Compute Z-scores for cleaned data
z_rawComb3 <- scale(Combined3)

# Flag outliers: values with |z| > 3
outlier_Comb3 <- abs(scale(Combined3))>3

# Count of outliers per variable
colSums(outlier_Comb3)
```

```
##  Y17 AR18  Y18  Y19 AR20  Y20
##    2    2    5    4    2    5
```

```
#head(Combined3)
nrow(Combined3)
```

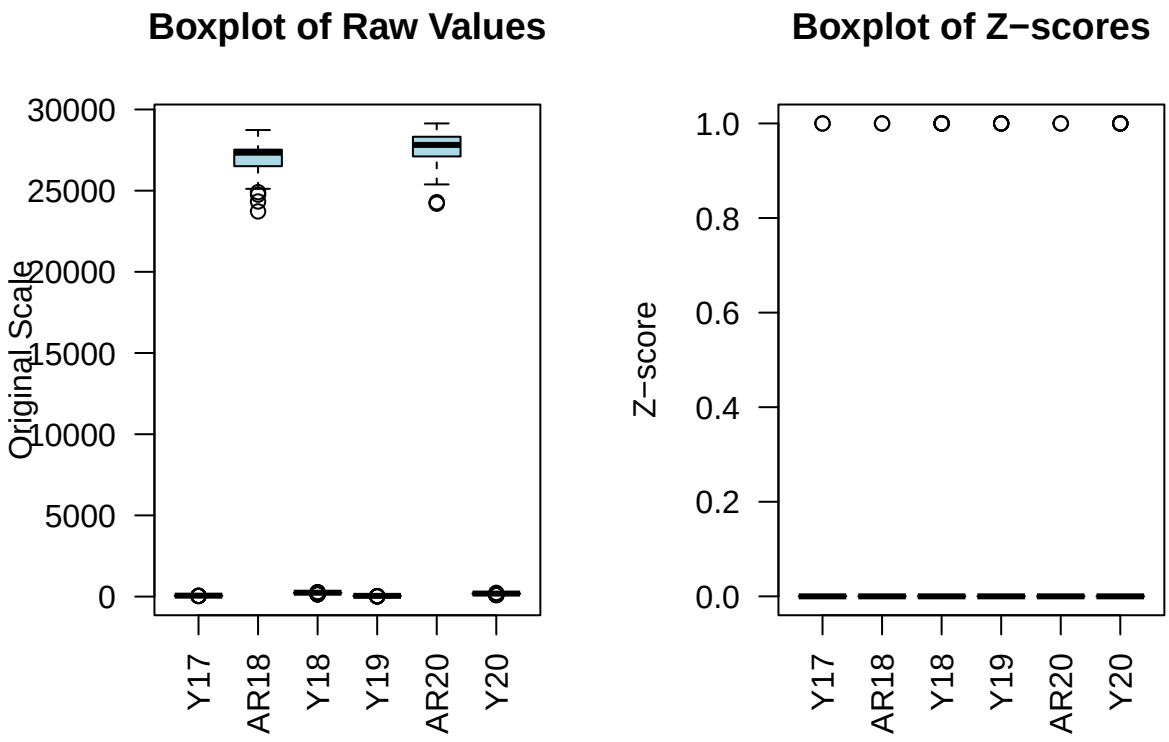
```
## [1] 202
```

Outlier Analysis: - Outliers are present across all years ranging from 2 to 5

- Y18 and Y20 (corn harvests yields) have the highest number of outliers, suggesting either possible variability in corn yields perhaps due to unusual environmental conditions or sensor error in these years.
- AR18 and AR20 (applied rates), have a lower cases of outliers, possibly due to machine adjustability, soil viability, or even the effect of fields' edges.
- From the data, we can conclude that the yield rates after 2017 show increased variations in outliers.

2. Boxplot: Raw Combined Data

```
# Set layout: 1 row, 2 plots
par(mfrow = c(1, 2))

# Boxplot of raw values
boxplot(Combined3,
        main = "Boxplot of Raw Values",
        ylab = "Original Scale",
        col = "lightblue",
        outline = TRUE,
        las = 2)

# Boxplot of standardized values (z-scores)
boxplot(outlier_Comb3,
        main = "Boxplot of Z-scores",
        ylab = "Z-score",
        col = "lightgreen",
        outline = TRUE,
        las = 2)
abline(h = c(-3, 3), col = "red", lty = 2)
```




```
# Reset layout
par(mfrow = c(1, 1))
```

Boxplots Analysis:

Raw Dataset Boxplot:

- Shows each variable in its real unit scale; hence the shown results - with the *AppliedRate* variable way above the *Yield* variable.
- The Median AppliedRate is around $27,500$ for Y18 and Y20 while all the yield variable appear with medians around 0.
- Whiskers and outliers depend on the spread of raw values.
- Care has to be taken in attempts to make comparison between the spread of these variables (AR and Y) since they are measured in different units and reference different crops

Scaled Data Analysis: - Standardizes all variables to mean = 0 and SD = 1.

- Helps detect which variables have extreme standardized deviations ($|z| > 3$).
- The outlier threshold, which are usually shown by Red dashed lines are invisible here.

3. Calculated Means, SD for Raw Data

```r
# Define variable names
vars <- c("Y17", "AR18", "Y18", "Y19", "AR20", "Y20")

# Calculate column means and standard deviations after normalization
norm_means <- colMeans(Combined3[, vars], na.rm = TRUE)
norm_sds   <- apply(Combined3[, vars], 2, sd, na.rm = TRUE)

# View resulting dataframe set of values
data.frame(Mean = round(norm_means, 3), SD = round(norm_sds, 3))
```

```
##           Mean      SD
## Y17     56.328   4.441
## AR18 26995.999 807.019
## Y18    231.198  23.576
## Y19     42.473   6.362
## AR20 27656.972 938.077
## Y20    181.521  20.992
```

Notes: This result gives a baseline snapshot of the raw dataset — the average values and their variability for each yield and applied rate over the years. It allows for comparison of the magnitude and spread of different variables before any transformation, helping guide normalization and modeling steps. When the data is transformed, $\mu \approx 0$ and $\sigma \approx 1$

Calculated Means, SD for Scaled Data

```r
# Define variable names
vars <- c("Y17", "AR18", "Y18", "Y19", "AR20", "Y20")

# Calculate column means and standard deviations after normalization
norm_means <- colMeans(z_rawComb3[, vars], na.rm = TRUE)
norm_sds   <- apply(z_rawComb3[, vars], 2, sd, na.rm = TRUE)

# View resulting dataframe set of values
data.frame(Mean = round(norm_means, 3), SD = round(norm_sds, 3))
```

```
##      Mean SD
## Y17     0  1
## AR18    0  1
## Y18     0  1
## Y19     0  1
## AR20    0  1
## Y20     0  1
```

Comparison Discussion Outliers of Combined Raw dataset vs Combined Normalized dataset:

- Which variables had many extreme values in raw or normalized form?

NORMALIZED DATA: - All scaled variables $> 3\sigma$ - outside the range $(\mu - 3\sigma, \mu + 3\sigma)$

- Effect of Normalization: Following normalization, all variables in the dataset show means close to zero and standard deviations near one. Boxplot visualizations confirm that each distribution has been centered and rescaled appropriately, enabling cross-variable comparisons and minimizing the influence of scale/unit disparities.

2. NORMALIZED DATA ANALYSIS

A. Normalization Process

To accomplish analysis on the same dataset, the same sequence of processing performed (file read, count requirement (>=30), calculaltion of mean & stand deviation), and plotting) on the raw dataset will be used here. Additionally, a method, referred to as the Winsorize method will be employed prior to the common method of normalizaling the data so that statistidal expedience could be brought to the process. Winsorization, is the process of trimming extreme tail values off datasets.

The data files contain three different measurement units for the data. Files `2017 Soybeans Harvest.csv` and `2019 Soybeans Harvest.csv` are soybean harvests datasets with around 60 bu/acre. `2018 Corn Harvest.csv` and `2020 Corn Harvest.csv` are corn harvests files with around 180 bu/acre, while `2018 Corn Seeding.csv` and `2020 Corn Seeding.csv` contain seeding rate data. Given that we are handling different crops, for different years, measuring in different units for different field locations, normalization is therefore required for a leveled analysis and to manage (lessen) lurking outliers. With the in-hand datasets, we will examine the distribution for outliers by comparing them for both the raw and normalized data by visualizing boxplots.

For normalization, the Z-Score approach will be used. Thereafter to fully ascertain a relationship between the variables, we will explore linear regression.

b. Rationale: Z-Score Normalization

Selection of the Z-Score normalization method based on:

- the dataset contains continuous variables *(Yield, AppliedRate)*
- the analysis depicts that comparison would be: across crops, years, and measurement units (soybeans vs. corn)
- requirement to perform *Bayesian Network Modeling* - the requirement to aggregate data spatially and compute cell-level means and counts
- the need to visualize with the data using plot pairs(), grided plots, heatmaps, and DAGs.

The collective requirement demands that a more vigorous stanbdardized method be employed for data scaling. Hence the selected of the Z-Score method. In comparison, the *RANK* method while robus to outliers and preserves ordering, it fails on the actual distance between non-metric values. Alternatively, using the *PERCENT* method keeps proportional meaning, and is simple to interpret, but is affected by extreme outliers.
We denote the i^{th} `Yield` observation for Year j as y_{ij}, we normalize yield by one of the following methods, in each case holding j constant and iterating over i only within years. If we assume 20 rows and 6 columns, then $y_{ij} = \left\{y_{1j}, y_{2j}, \dots, y_{N_i j}\right\}$ where $N_i = 120$. Similarly, we would denote the successive yield estimates for grid cell i as $y_{ij} = \left\{y_{i1}, y_{i2}, \dots, y_{iN_j}\right\}$ where $N_j = 5$.

In short, *Z-score normalization* was selected for its ability to center the data and standardize the variance across different units (e.g., yield vs. seeding rate). This ensures comparability across crop types and years while mitigating the influence of extreme values on downstream causal inferences.

B. FORMULA: Z-score Normalization Method

Calculate:

$$\overline{y}_{.j} = \frac{\sum_{i=1}^{N_i} y_{ij}}{N_i}$$

and

$$s_{.j}^2 = \frac{\sum_{i=1}^{N_i} (y_{ij} - \overline{y}_{.j})^2}{N_i - 1}$$

where N_i are the number of `Yield` values for year j. Replace y_{ij} with

$$z_{ij} = \frac{y_{ij} - \overline{y}_{.j}}{s_{.j}}$$

.

We calculate $\overline{y}_{.j}$ and $s_{.j}^2$ independently for $j = 1, 2, \dots, N_j$ for each original data column. Note that this method makes use of the first (mean) and second moments (variance). It would be best practice to check for skewness or kurtosis of the data.

Called Aggregation Function

```r
aggregate_by_grid <- function(data, value_col, cell_size = 50) {
  # Compute Row, Column, and Cell
  data$Row <- ceiling(data$Latitude / cell_size)
  data$Column <- ceiling(data$Longitude / cell_size)
  data$Cell <- data$Row * 1000 + data$Column

  # Subset to keep only needed columns
  df_sub <- data.frame(Cell = data$Cell,
                       Row = data$Row,
                       Column = data$Column,
                       Latitude = data$Latitude,
                       Longitude = data$Longitude,
                       Value = data[[value_col]])

  # Compute mean per grid cell
  agg_mean <- aggregate(Value~Cell+Row+Column, data=df_sub, FUN=mean, na.rm=TRUE)
  colnames(agg_mean)[4] <- "Mean"

  # Compute count per grid cell
  agg_count <- aggregate(Value~Cell+Row+Column, data=df_sub, FUN=length)
  colnames(agg_count)[4] <- "Count"

  # Step 5: Merge mean and count results
  agg_result <- merge(agg_mean, agg_count, by=c("Cell", "Row", "Column"))

  return(agg_result)
}
```

C. Normalized Data Strategy

1. Data Input - Read Files

```r
# Load data and drop unnamed first column from files
soyHarvest_2017 <- read.csv("Data/2017 Soybeans Harvest.csv")[, -1]
cornSeed_2018 <- read.csv("Data/2018 Corn Seeding.csv")[, -1]
cornHarvest_2018 <- read.csv("Data/2018 Corn Harvest.csv")[, -1]
soyHarvest_2019 <- read.csv("Data/2019 Soybeans Harvest.csv")[ ,-1]
cornSeed_2020 <- read.csv("Data/2020 Corn Seeding.csv")[, -1]
cornHarvest_2020 <- read.csv("Data/2020 Corn Harvest.csv")[, -1]

# Aggregate by Yield/Applied Rate
agg_grid_y17 <- aggregate_by_grid(soyHarvest_2017, "Yield")
colnames(agg_grid_y17)[4] <- "Y17"
#head(agg_grid_y17)

agg_grid_ar18 <- aggregate_by_grid(cornSeed_2018, "AppliedRate")
colnames(agg_grid_ar18)[4] <- "AR18"

agg_grid_y18  <- aggregate_by_grid(cornHarvest_2018,  "Yield")
colnames(agg_grid_y18)[4] <- "Y18"
```

```
agg_grid_y19  <- aggregate_by_grid(soyHarvest_2019,   "Yield")
colnames(agg_grid_y19)[4] <- "Y19"

agg_grid_ar20 <- aggregate_by_grid(cornSeed_2020,     "AppliedRate")
colnames(agg_grid_ar20)[4] <- "AR20"

agg_grid_y20  <- aggregate_by_grid(cornHarvest_2020,  "Yield")
colnames(agg_grid_y20)[4] <- "Y20"
```

Data File Reads

```
files <- c("Data/2017 Soybeans Harvest.csv",
           "Data/2018 Corn Seeding.csv",
           "Data/2018 Corn Harvest.csv",
           "Data/2019 Soybeans Harvest.csv",
           "Data/2020 Corn Seeding.csv",
           "Data/2020 Corn Harvest.csv")

# columns captured to be aligned with the file read
value_cols <- c("Yield", "AppliedRate", "Yield", "Yield", "AppliedRate", "Yield")
years <- c(2017, 2018, 2018, 2019, 2020, 2020)

# In case need arise for analysis for Harvest or Seeding
types <- c("SoyHarvest", "CornSeed", "CornHarvest", "SoyHarvest", "CornSeed", "CornHarvest")
```

2. Data File Processing

```
process_file <- function(filepath, value_col, year, type, cell_size = 50) {
  df <- read.csv(filepath)

  if ("X" %in% colnames(df)) df <- df[, !(colnames(df) == "X")]

  df$Year <- year
  df$Type <- type
  df$Variable <- value_col # Variable of type yield/appliedrate

  # Rename target value column as "Value" xxxx
  if (!("Value" %in% colnames(df)) && value_col %in% colnames(df)) {
    colnames(df)[colnames(df) == value_col] <- "Value"
  }

  df$Row <- ceiling(df$Latitude / cell_size)
  df$Column <- ceiling(df$Longitude / cell_size)
  df$Cell <- df$Row * 1000 + df$Column

  # Ensure all required columns exist before subsetting
  needed_cols <- c("Year", "Latitude", "Longitude", "Row", "Column", "Cell", "Variable", "Type", "Value

  missing_cols <- setdiff(needed_cols, colnames(df))
```

```
  if (length(missing_cols) > 0) {
    stop(paste("Missing columns in", filepath, ":", paste(missing_cols, collapse = ", ")))
  }

  df_sub <- df[, needed_cols]
  return(df_sub)
}
```

```
filtered_list <- list()

for (i in seq_along(files)) {

  df <- process_file(files[i], value_cols[i], years[i], types[i])

  cell_counts <- table(df$Cell)
  valid_cells <- names(cell_counts[cell_counts >= 30]) #valid cnt value
  df_filtered <- df[df$Cell %in% valid_cells, ]

  # list contains multiple colums
  filtered_list[[i]] <- df_filtered
}
```

3. Data Selection

```
# want to ensure atleast 30 observations prior to merge
filter_cells_base <- function(df, min_obs = 30) {
  cell_counts <- table(df$Cell)
  valid_cells <- names(cell_counts[cell_counts >= min_obs])
  df_filtered <- df[df$Cell %in% valid_cells, ]
  #df_filtered <- na.omit(df_filtered)
  #df_filtered <- df_filtered[df_filtered$Yield > 0, ]
  #df_filtered <- df_filtered[df_filtered$AppliedRate >= 0, ]
  return(df_filtered)
}

# Apply filter_cell_base to all processed datasets
filtered_datasets <- lapply(filtered_list, filter_cells_base)
```

4. Data Merge

```
library(data.table)

# merge files
# Combined_filtered <- do.call(rbind, filtered_list)
merged_data <- rbindlist(filtered_datasets, fill = TRUE)
```

```
dim(merged_data)        # Rows and columns
```

```
## [1] 109702      9
```

```
#table(merged_data$Year) # Count of rows by year
#head(merged_data)       # Preview
```

```
# Check how many rows survived filtering in each dataset
sapply(filtered_datasets, nrow)
```

```
## [1] 21382  8841 25087 20738  9105 24549
```

```
# Makes copy of merged raw dataset; referenced as combined_data thereafter
combined_data <- merged_data
head(combined_data)
```

```
##     Year    Latitude Longitude   Row Column  Cell Variable       Type Value
##    <num>       <num>     <num> <num>  <num> <num>   <char>     <char> <num>
## 1:  2017 0.002211722  539.2350     1     11  1011    Yield SoyHarvest     0
## 2:  2017 0.088358278  539.2586     1     11  1011    Yield SoyHarvest     0
## 3:  2017 0.556358615  539.4221     1     11  1011    Yield SoyHarvest     0
## 4:  2017 1.362200247  539.8540     1     11  1011    Yield SoyHarvest     0
## 5:  2017 2.289686799  540.5014     1     11  1011    Yield SoyHarvest     0
## 6:  2017 3.289275652  541.3224     1     11  1011    Yield SoyHarvest     0
```

```
#tail(combined_data)
#nrow(combined_data)
```

D. ALGORITHM - Project Approach

The same as used in Raw Data solution with the extension of applying the Winsorize function followed by the Z-Scaling funtion.

1. Winsorize & Normalized Functions

Simple trimming of values from both extrememes (5% lower and 5% upper) of dataset to be followed by Z-scale normalization. Here, values less than 0.05 will be capped to the 0.05 value; values exceeding the 0.95% value will be capped at 0.95.We note that trimming can take more aggressive actions, but it is hoped that this interval will suffice.

```
# winsorize_base function
winsorize_base <- function(x, lower = 0.05, upper = 0.95) {
  nonzero_x <- x[x != 0]
  qnt <- quantile(nonzero_x, probs = c(lower, upper), na.rm = TRUE)
  x_winsor <- x
  x_winsor[x > qnt[2] & x != 0] <- qnt[2]
  x_winsor[x < qnt[1] & x != 0] <- qnt[1]
  return(x_winsor)
}
```

2. Z-Norm function

```r
# normalize_vector function
normalize_vector <- function(x) {
  mu <- mean(x, na.rm = TRUE)
  sigma <- sd(x, na.rm = TRUE)
  (x - mu) / sigma
}
```

Winsorize Function

```r
# process_all_vars function
process_vars <- function(dataset, lowers = c(0.05, 0.10), uppers = c(0.95, 0.90)) {
  vars <- unique(dataset$Variable)
  result_list <- list()

  for (var in vars) {
    df_var <- dataset[dataset$Variable == var, ]
    x <- df_var$Value
    for (i in seq_along(lowers)) {
      lower <- lowers[i]
      upper <- uppers[i]
      wins_col <- paste0("wins_", lower*100, "_", upper*100)
      df_var[[wins_col]] <- winsorize_base(x, lower=lower, upper=upper)
      norm_col <- paste0("z_", lower*100, "_", upper*100)
      df_var[[norm_col]] <- normalize_vector(df_var[[wins_col]])
    }
    result_list[[var]] <- df_var
  }
  combined_result <- do.call(rbind, result_list)
  rownames(combined_result) <- NULL
  return(combined_result)
}
```

```r
processed_data <- process_vars(combined_data)

# Reassign winsorized data to Combined_data
Combined_data  <- processed_data
```

```r
head(processed_data)
```

```
##     Year    Latitude Longitude   Row Column  Cell Variable       Type Value
##    <num>       <num>     <num> <num>  <num> <num>   <char>     <char> <num>
## 1:  2017 0.002211722  539.2350     1     11  1011    Yield SoyHarvest     0
## 2:  2017 0.088358278  539.2586     1     11  1011    Yield SoyHarvest     0
## 3:  2017 0.556358615  539.4221     1     11  1011    Yield SoyHarvest     0
## 4:  2017 1.362200247  539.8540     1     11  1011    Yield SoyHarvest     0
## 5:  2017 2.289686799  540.5014     1     11  1011    Yield SoyHarvest     0
## 6:  2017 3.289275652  541.3224     1     11  1011    Yield SoyHarvest     0
##    wins_5_95    z_5_95 wins_10_90   z_10_90
##        <num>     <num>      <num>     <num>
## 1:         0 -1.556245          0 -1.581877
```

```
## 2:         0 -1.556245          0 -1.581877
## 3:         0 -1.556245          0 -1.581877
## 4:         0 -1.556245          0 -1.581877
## 5:         0 -1.556245          0 -1.581877
## 6:         0 -1.556245          0 -1.581877
```

3. Mean and Count for Scaled Dataset

```r
# Get unique variable names
vars <- unique(Combined_data$Variable)

# Initialize empty list to collect results
results <- list()

# Loop over each variable
for (v in vars) {
  df <- Combined_data[Combined_data$Variable == v, ]

  stats <- c(
    # mean and sd for winsorized (trimmed) dataset
    mean_wins_05_95 = mean(as.numeric(df$wins_5_95), na.rm = TRUE),
    sd_wins_05_95   = sd(as.numeric(df$wins_5_95), na.rm = TRUE),

    # mean and sd for scaled (z-norm) dataset
    mean_z_5_95    = mean(as.numeric(df$z_5_95), na.rm = TRUE),
    sd_z_5_95      = sd(as.numeric(df$z_5_95), na.rm = TRUE)
  )

  results[[v]] <- stats
}

# Convert list to a data frame
summary_table <- do.call(rbind, results)
summary_table <- data.frame(Variable = rownames(summary_table), summary_table, row.names = NULL)

# View
print(summary_table)
```

```
##      Variable mean_wins_05_95 sd_wins_05_95  mean_z_5_95 sd_z_5_95
## 1       Yield        131.2448       84.3343 1.349548e-16         1
## 2 AppliedRate      27362.1128     1046.3705 2.801076e-17         1
```

```r
# Raw means and SDs for Yield and AppliedRate
raw_stats <- aggregate(Value ~ Variable,
                       data = Combined_data[Combined_data$Variable %in% c("Yield", "AppliedRate"), ],
                       FUN = function(x) c(mean = mean(x, na.rm = TRUE), sd = sd(x, na.rm = TRUE)))

# Clean up result
raw_df <- do.call(data.frame, raw_stats)
names(raw_df) <- c("Variable", "Mean_raw", "SD_raw")

print(raw_df)
```

```
##      Variable  Mean_raw     SD_raw
## 1 AppliedRate 27305.379 1581.88188
## 2       Yield   132.908   96.49688
```

4. Data Selection

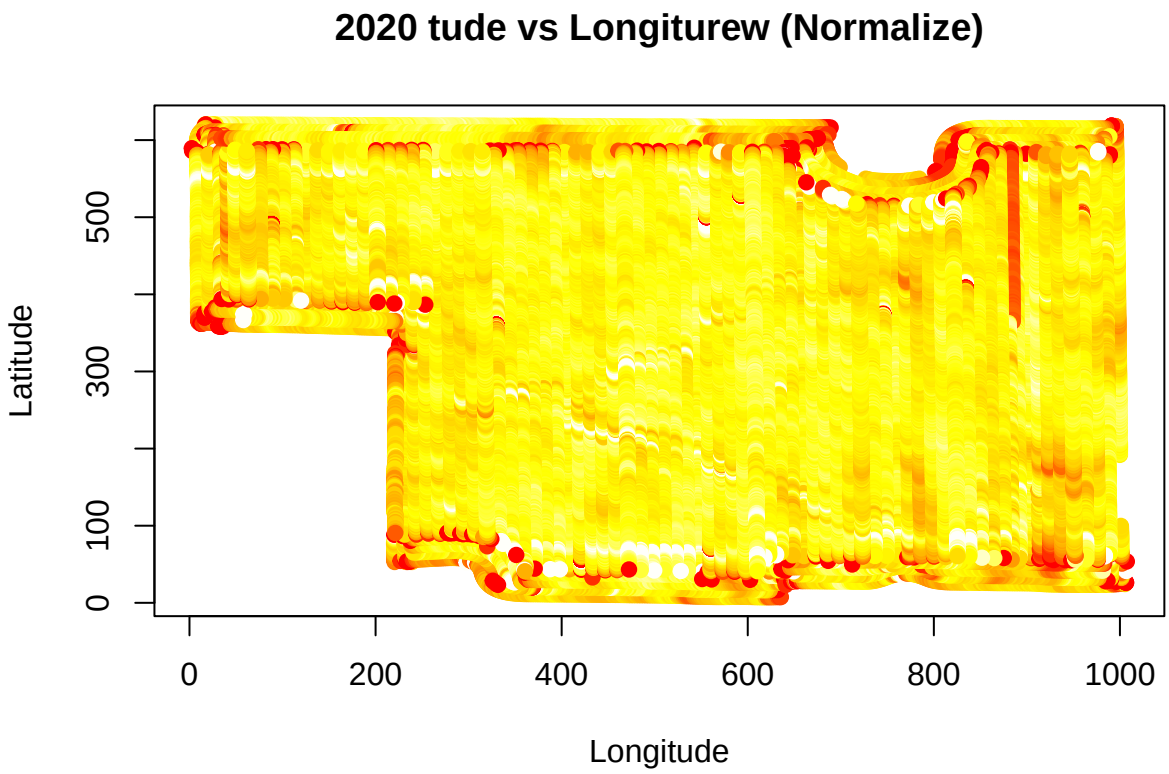
```r
# Compute count of rows per Cell
cell_counts <- table(Combined_data$Cell)

# Find Cells with at least 30 observations
valid_cells <- as.numeric(names(cell_counts[cell_counts >= 30]))

# Subset the data to keep only VaLid Cells
Combined_filtered <- Combined_data[Combined_data$Cell %in% valid_cells, ]

# View result
cat("Number of valid rows retained:", nrow(Combined_filtered), "\n")
```

```
## Number of valid rows retained: 109702
```

E. OUTPUT (Winsorized + Normalized Data)

1. Simple Plot: Normalized 2020 Harvest Data

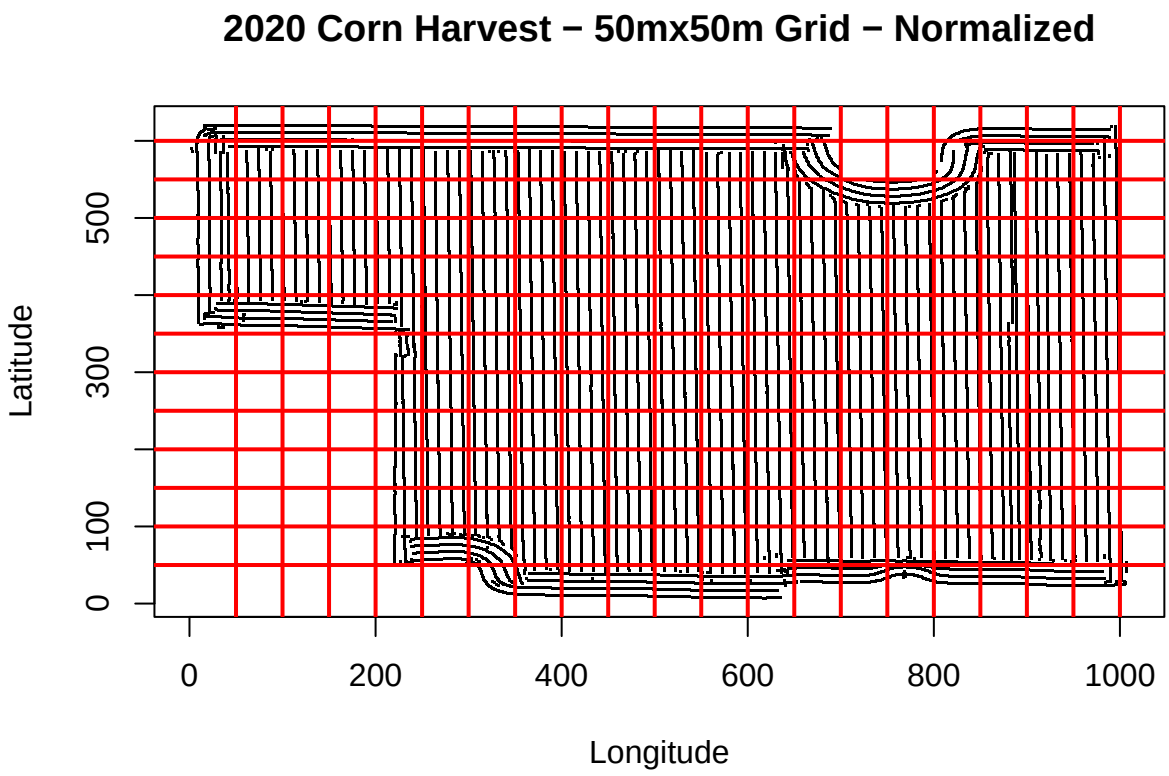
```r
# Filter for 2020 Harvest data only
harvest_2020 <- Combined_data[Combined_data$Year == 2020 & Combined_data$Type == "CornHarvest", ]

# Remove rows with missing Longitude, Latitude, or z_5_95
harvest_2020 <- harvest_2020[!is.na(harvest_2020$Longitude) & !is.na(harvest_2020$Latitude) & !is.na(ha

# If no data, stop
if (nrow(harvest_2020) == 0) {
  stop("No valid 2020 Harvest data to plot.")
}

# Define colors based on normalized z_5_95 values
colors <- heat.colors(100)
scaled_vals <- as.numeric(cut(harvest_2020$z_5_95, breaks = 100))

# Plot with colors representing z_5_95
plot(harvest_2020$Longitude, harvest_2020$Latitude,
     col = colors[scaled_vals],
     pch = 19,
     xlab = "Longitude",
     ylab = "Latitude",
     main = "2020 tude vs Longiturew (Normalize)")
```



2. 2020 CornHarvest Spatial Grid Format (50mx50m)

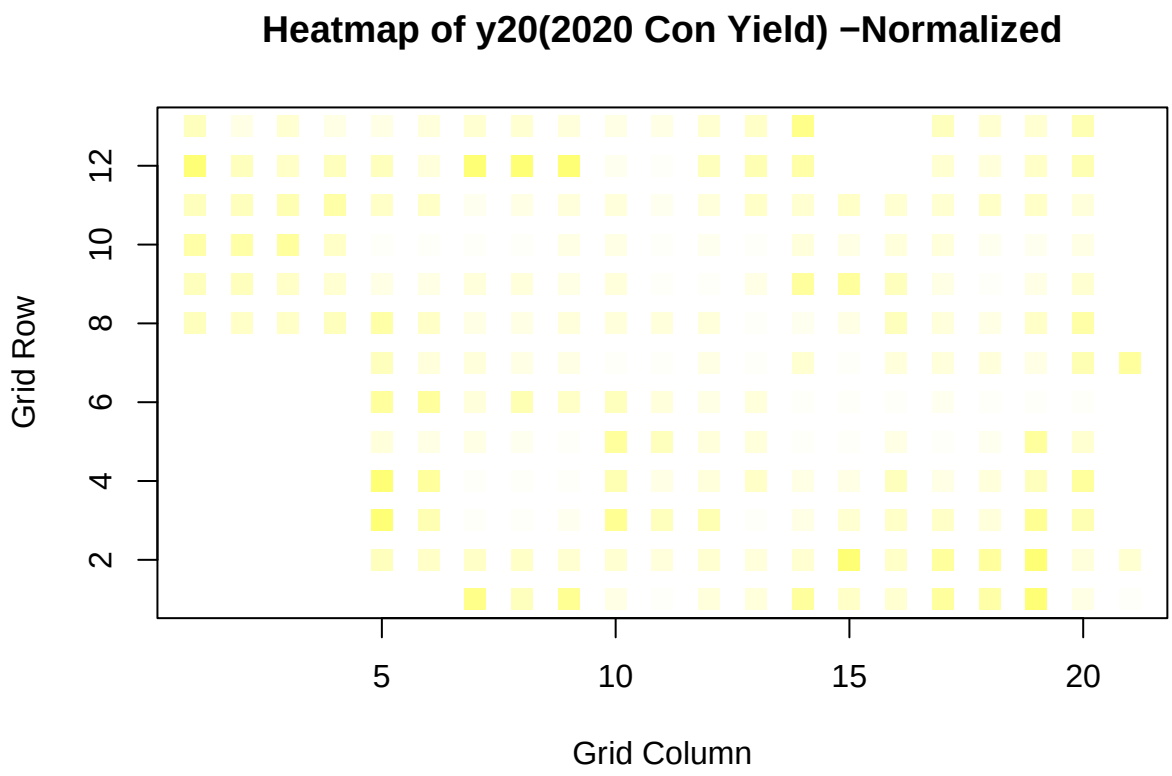
```r
# Subset 2020 harvest data (adjust 'Type' as needed)
harvest_2020 <- Combined_data[Combined_data$Year == 2020 & Combined_data$Type == "CornHarvest", ]

# Create Row and Column variables based on 50m grid using ceiling(Latitude/50) and ceiling(Longitude/50)
harvest_2020$Row <- ceiling(harvest_2020$Latitude / 50)
harvest_2020$Column <- ceiling(harvest_2020$Longitude / 50)

# Create Cell identifier (Row * 1000 + Column)
harvest_2020$Cell <- harvest_2020$Row * 1000 + harvest_2020$Column

# Aggregate mean per cell
agg_mean <- aggregate(Value ~ Row + Column + Cell, data = harvest_2020, FUN = mean, na.rm = TRUE)

# Aggregate count per cell
agg_count <- aggregate(Value ~ Row + Column + Cell, data = harvest_2020, FUN = length)

# Rename columns for clarity
names(agg_mean)[names(agg_mean) == "Value"] <- "Mean"
names(agg_count)[names(agg_count) == "Value"] <- "Count"

# Merge mean and count
agg_data <- merge(agg_mean, agg_count, by = c("Row", "Column", "Cell"))
```

```r
# Plot Latitude ~ Longitude with grid lines every 50 units
plot(harvest_2020$Longitude, harvest_2020$Latitude, pch = ".", xlab = "Longitude", ylab = "Latitude", ma
abline(h = (1:12)*50, v = (1:20)*50, col = "red", lwd = 2)
```



3. Heatmap of 2020 Corn Yield

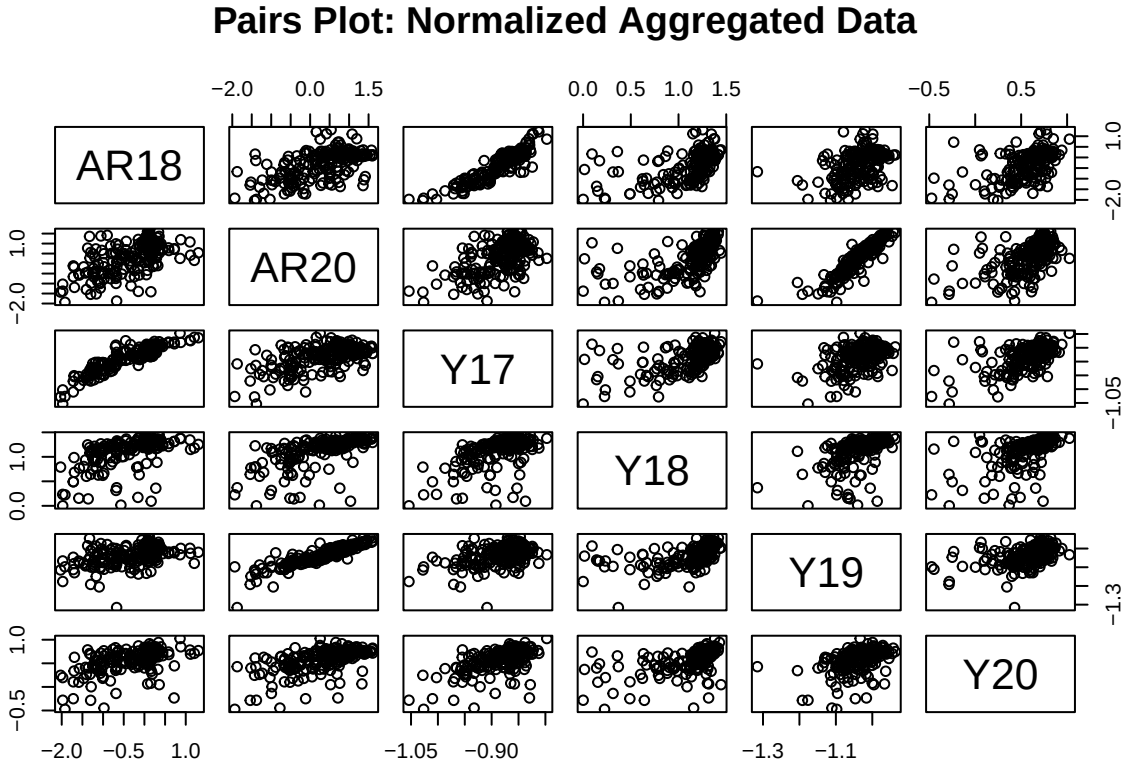
```r
# Corrected plotting code
plot_data_z <- Combined_data[!is.na(Combined_data$z_5_95), ]

z_vals <- as.numeric(plot_data_z$z_5_95)

num_colors <- 100
color_bins_z <- cut(z_vals, breaks = num_colors)
colors_z <- heat.colors(num_colors)[as.numeric(color_bins_z)]

plot(plot_data_z$Column, plot_data_z$Row,
     col = colors_z, pch = 15, cex = 1.5,
     xlab = "Grid Column", ylab = "Grid Row",
     main = "Heatmap of y20(2020 Con Yield) -Normalized")
```



4. Pairwise Relations: Yield & Seeding Rates (2017-2020)

```
# Create VarYear with labels like Y17, AR18, H20, S19
Combined_data$VarYear <- with(Combined_data, {
  abbrev <- ifelse(Variable == "Yield", "Y",
                   ifelse(Variable == "AppliedRate", "AR",
                   ifelse(Type == "Harvest", "H",
                   ifelse(Type == "Seeding", "S", NA))))
  year_short <- substr(Year, 3, 4)
  paste0(abbrev, year_short)
})

# Aggregate: compute mean z_5_95 value per Cell and VarYear
agg_data <- aggregate(z_5_95 ~ Row + Column + VarYear,
                      data = Combined_data, FUN = mean)

# Reshape from long to wide
wide_data <- reshape(data = agg_data,
                     idvar = c("Row", "Column"),
                     timevar = "VarYear",
                     direction = "wide")

# Clean column names
names(wide_data) <- sub("z_5_95\\.", "", names(wide_data))
```

```r
# Remove rows with NAs
clean_data <- na.omit(wide_data)

# Create pairs plot
pairs(clean_data[, -c(1, 2)],
      main = "Pairs Plot: Normalized Aggregated Data")
```



5. Visualized Combined Normalized Data View

```r
create_wide_summary <- function(data, value_col, summary_fun = mean, ...) {
  # Ensure the value column exists
  if (!value_col %in% names(data)) {
    stop(paste("Column", value_col, "not found in data."))
  }

  # Dynamically create the formula: value_col ~ Row + Column + VarYear
  formula_str <- paste(value_col, "~ Row + Column + VarYear")
  formula_obj <- as.formula(formula_str)

  # Aggregate the data
  agg_data <- aggregate(formula_obj,
                        data = data,
                        FUN = summary_fun,
```

```
                        ...)

  # Reshape to wide format
  wide_data <- reshape(agg_data,
                       idvar = c("Row", "Column"),
                       timevar = "VarYear",
                       direction = "wide")

  # Sort value columns by year suffix
  id_cols <- c("Row", "Column")
  value_cols <- setdiff(names(wide_data), id_cols)

  # Extract year suffix (17, 18,...) from column names and sort
  sorted_value_cols <- value_cols[order(as.numeric(sub(".*\\.", "", value_cols)))]

  # Create numeric Cell ID
  wide_data$Cell <- wide_data$Row * 1000 + wide_data$Column

  # Reorder: place Cell followed by sorted values
  wide_data <- wide_data[, c("Cell", sorted_value_cols)]

  return(wide_data)
}
```

Notes: Sorted values in order: (AR17, AR18, Y17, etc.)

Visualized Merge Normalized Dataset

```
wide_z5_95 <- create_wide_summary(Combined_data, "z_5_95", mean, na.rm = TRUE)
```

```
## Warning in eval(quote(list(...)), env): NAs introduced by coercion
```

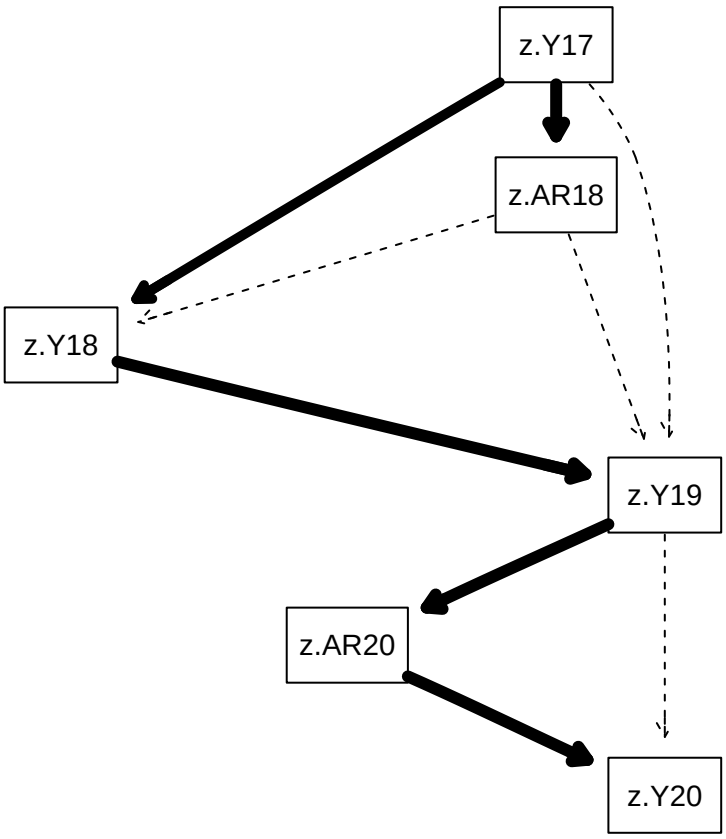
```
head(wide_z5_95)
```

```
##    Cell z_5_95.AR18 z_5_95.AR20 z_5_95.Y17 z_5_95.Y18 z_5_95.Y19 z_5_95.Y20
## 1  8001  -1.9213284  -1.9358509 -1.0265580  0.2246157  -1.097718 -0.4744617
## 2  9001  -0.7706741  -0.4971648 -0.9330765  0.8997355  -1.085984  0.5122085
## 3 10001  -1.2134839  -0.4323335 -0.9234435  0.9534400  -1.061734  0.4014553
## 4 11001  -0.8111415  -0.7616415 -0.9247440  0.8415962  -1.102783  0.6354213
## 5 12001  -0.9813489  -0.4879879 -0.9503524  0.6643643  -1.075339  0.4658139
## 6  8002  -1.3596746  -0.4174676 -0.9653397  0.1416735  -1.060955  0.3992695
```

###6. Causual Analysis - DAG for SCALED DATASET Since DAGs only show data dependencies, note is made that DAGs using normalized data would reveal similar strengths as those for the raw data presented earlier. will only produce a full model DAG (2017-2020) here, using our normalized dataset.

i. DAG model for 2017 - 2020

```r
library(bnlearn)
library(Rgraphviz)

# Rename columns in wide_z5_95 just once
colnames(wide_z5_95) <- sub("^z_5_95\\.", "z.", colnames(wide_z5_95))

# Prepare modeling data
model_data_z <- wide_z5_95[, c("z.Y17", "z.AR18", "z.Y18", "z.Y19", "z.AR20", "z.Y20")]
model_data_z <- na.omit(model_data_z)

#"Cell"   "z.AR18" "z.AR20" "z.Y17"  "z.Y18"  "z.Y19"  "z.Y20"

# Regression Model created
#model_Y20_z <- lm(z.Y20 ~ z.Y17 + z.Y18 + z.Y19 + z.AR18 + z.AR20, data = #model_data_z)
#summary(model_Y20_z)

modela.dag <- model2network("[z.Y17][z.AR18|z.Y17][z.Y18|z.AR18:z.Y17]")

# Subset to match DAG variables
data_a <- model_data_z[, c("z.Y17", "z.AR18", "z.Y18")]
# Fit and plot
fita <- bn.fit(modela.dag, data_a)
strengtha <- arc.strength(modela.dag, data_a)
#strength.plot(modela.dag, strengtha)

# Define DAG
modelb.dag <- model2network("[z.Y19][z.AR20|z.Y19][z.Y20|z.AR20:z.Y19]")

# Subset data to include only the nodes in the DAG
data_b <- model_data_z[, c("z.Y19", "z.AR20", "z.Y20")]

# Fit and plot
fitb <- bn.fit(modelb.dag, data_b)
strengthb <- arc.strength(modelb.dag, data_b)
#strength.plot(modelb.dag, strengthb)


# DAG c - representing combination
#modelc.dag <- model2network("[z.Y17][z.AR18|z.Y17][z.Y18|z.AR18:z.Y17][z.Y19|z.Y17:z.AR18:z.Y18][z.AR2
modelc.dag <- model2network("[z.Y17][z.AR18|z.Y17][z.Y18|z.AR18:z.Y17][z.Y19|z.Y17:z.AR18:z.Y18][z.AR20
fitc <- bn.fit(modelc.dag, model_data_z)
strengthc <- arc.strength(modelc.dag, model_data_z)
strength.plot(modelc.dag, strengthc)
```




```
#sort(nodes(modelc.dag))
```

```
#sort(nodes(modelc.dag)) == sort(colnames(model_data_z))
```

7. DAG COMPARISON - RAW (below) vs NORMALIZED (above)

ANALYSIS: We observe that the structure of the DAG of the Raw dataset (visual at $6.f.iii$) and that of the normalized dataset are similar. The same conclusion can therefore be drawn - that the yield rate of 2017 is a good predictor for the yield rate of 2020. Hence, **normalization preseved the structure of a DAG.**

Will proceed to test the strength of these links to fully verify our project's objective: whether a relationship exists between the *yield and the applied seeding and/or yield (previous years).*

Calculated Strengths of DAG: 2017-2020

```
library(bnlearn)

model_data_raw <- Combined3[, c("Y17", "Y18", "Y20", "AR18")]
model_data_raw <- na.omit(model_data_raw)

# Learn the network structure
modelc.dag <- hc(model_data_raw)
```

```r
# Compute arc strength values using test-based criterion RAW
Rstrength_df <- arc.strength(modelc.dag, data = model_data_raw, criterion = "cor")

# View the first few rows
# head(Rstrength_df)

# Sort the  strengths
Rsorted_strengths<- Rstrength_df[order(-Rstrength_df$strength), ]

Rsorted_strengths
```

7a. Arc Strengths of Raw Data

```
##   from   to     strength
## 5 AR18  Y20 4.768041e-03
## 4  Y18  Y20 1.034353e-06
## 3  Y17  Y20 5.584207e-08
## 2  Y17  Y18 7.614679e-24
## 1  Y17 AR18 6.864785e-75
```

7b. Arc Strengths of Normalized Data

```r
library(bnlearn)

# Learn the network structure
modelc.dag <- hc(model_data_z)

# Compute arc strength values using test-based criterion
Nstrength_df <- arc.strength(modelc.dag, data = model_data_z, criterion = "cor")

# View the first few rows
#head(Nstrength_df)

# Sort the  strengths
Nsorted_strengths <- Nstrength_df[order(-Nstrength_df$strength), ]

Nsorted_strengths
```

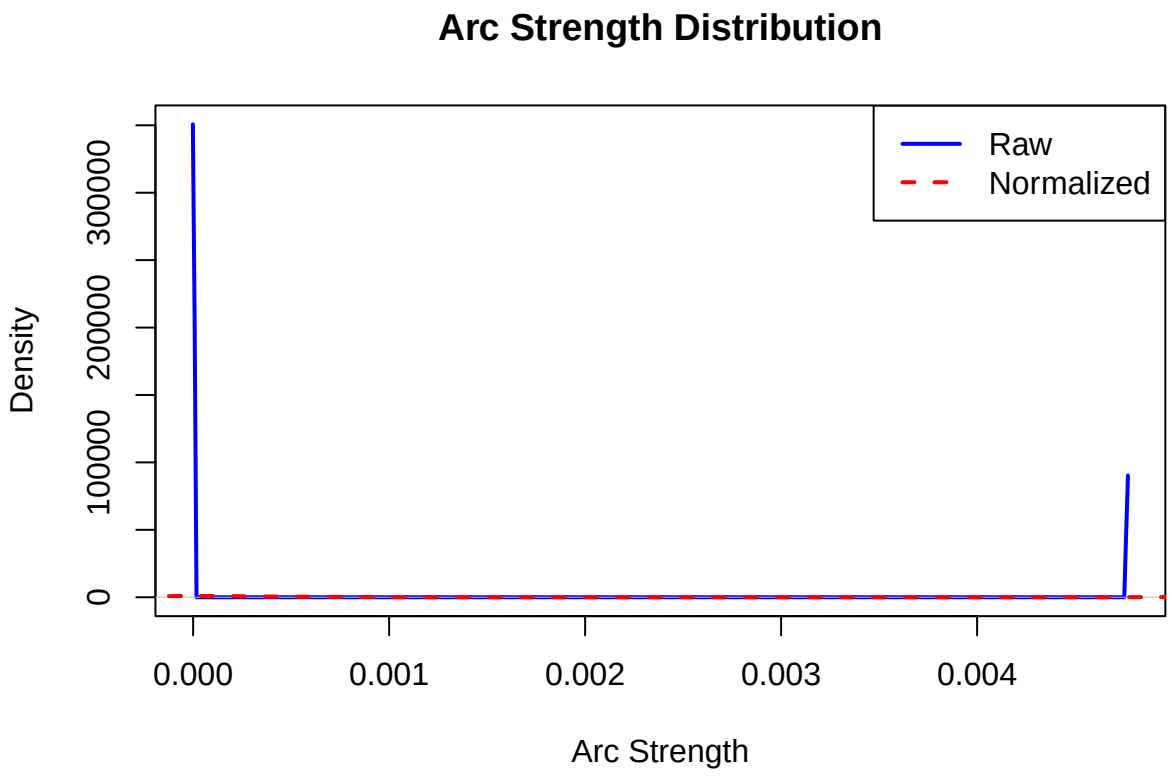
```
##      from     to     strength
## 10  z.Y19 z.AR18 1.817041e-02
## 8   z.Y18  z.Y20 5.103262e-03
## 9  z.AR20 z.AR18 6.818397e-04
## 6  z.AR20  z.Y20 3.973602e-04
## 4   z.Y17  z.Y20 8.038124e-07
## 7  z.AR20  z.Y17 6.023703e-07
## 5   z.Y18  z.Y17 2.884985e-07
## 3  z.AR20  z.Y18 3.276752e-22
## 1   z.Y17 z.AR18 6.654734e-58
## 2   z.Y19 z.AR20 1.119163e-75
```

Arc Length Comparisons:

- Varying number of reporting arc strengths for the Raw (5) and Normal (10) datasets.
- In the Raw dataset, the strongest link is between Y18 and Y20 ($4.768041e-08$) while that for the Normalized dataset is between zY19 and zAR20 ($1.817041e-02$. We note too the difference in magnitude in the top level supporting links.
- In the Raw dataset, the weakest link is between Y17 and AR18 ($6.864785e-75$) while that for the Normalized dataset is between zY19 and zAR18 ($1.119163e-75$) . We note too the difference in magnitude in the bottom level supporting links.
- Numerically, though, the Normalized dataset has the highest strength
- The normalized model retains and clarifies the core structural relationships among key agricultural variables (e.g., yield, applied rate across years). The arc strengths, especially the top quartile, reinforce the robustness of these relationships. Overall, the normalization process has enhanced interpretability and reduced scale-driven bias, making the Bayesian network structure more stable for downstream inference.

7c: Arc Length Density plots & Summary Stats

```
# Load bnlearn
library(bnlearn)

model_data_raw <- Combined3[, c("Y17", "Y18", "Y20", "AR18")]
model_data_raw <- na.omit(model_data_raw)

# Compute arc strengths of both moderls
arc_raw <- arc.strength(hc(model_data_raw), data = model_data_raw, criterion = "cor")
arc_norm <- arc.strength(hc(model_data_z), data = model_data_z, criterion = "cor")

# Add source column manually
arc_raw$Source <- "Raw"
arc_norm$Source <- "Normalized"

# Combine both datasets
arc_combined <- rbind(arc_raw, arc_norm)

# Set up the plot layout
par(mfrow = c(1, 1))  # 1 panel

# Density plots
plot(density(arc_raw$strength),
     col = "blue", lwd = 2, main = "Arc Strength Distribution",
     xlab = "Arc Strength", ylim = c(0, max(density(arc_raw$strength)$y, density(arc_norm$strength)$y)))
lines(density(arc_norm$strength), col = "red", lwd = 2, lty = 2)
legend("topright", legend = c("Raw", "Normalized"), col = c("blue", "red"), lwd = 2, lty = c(1, 2))
```




```r
# Descriptive statistics for each datasets
summary_raw <- c(
  Mean = mean(arc_raw$strength),
  Median = median(arc_raw$strength),
  SD = sd(arc_raw$strength),
  Min = min(arc_raw$strength),
  Max = max(arc_raw$strength),
  Count = length(arc_raw$strength)
)

summary_norm <- c(
  Mean = mean(arc_norm$strength),
  Median = median(arc_norm$strength),
  SD = sd(arc_norm$strength),
  Min = min(arc_norm$strength),
  Max = max(arc_norm$strength),
  Count = length(arc_norm$strength)
)

# Print summary stats side by side
comparison_table <- rbind(Raw = summary_raw, Normalized = summary_norm)
print(round(comparison_table, 4))
```

```
##                Mean Median     SD Min    Max Count
## Raw          0.0010      0 0.0021   0 0.0048     5
## Normalized 0.0024      0 0.0057   0 0.0182    10
```

```r
# Fix the column names of the normalized dataset to match raw
colnames(model_data_z) <- gsub("^z\\.", "", colnames(model_data_z))

# Learn network structures
fit_raw <- hc(model_data_raw)
fit_norm <- hc(model_data_z)

# Compute arc strengths
raw_strength <- arc.strength(fit_raw, data = model_data_raw)
norm_strength <- arc.strength(fit_norm, data = model_data_z)

# Create arc_id with sorted node names
raw_strength$arc_id <- apply(raw_strength[, c("from", "to")], 1, function(x) paste(sort(x), collapse =
norm_strength$arc_id <- apply(norm_strength[, c("from", "to")], 1, function(x) paste(sort(x), collapse =

# Merge arc strengths by arc_id
merged_arcs <- merge(
  raw_strength[, c("arc_id", "strength")],
  norm_strength[, c("arc_id", "strength")],
  by = "arc_id",
  suffixes = c("_raw", "_norm")
)

# Reconstruct readable arc labels
merged_arcs$arc <- gsub("_", "   ", merged_arcs$arc_id)

# Sort by raw strength
merged_arcs <- merged_arcs[order(-merged_arcs$strength_raw), ]

# View result
print(merged_arcs)
```

```
##     arc_id strength_raw strength_norm        arc
## 4  Y18_Y20    -9.540109     -1.347586  Y18   Y20
## 3  Y17_Y20   -12.423925     -9.788422  Y17   Y20
## 2  Y17_Y18   -48.658839    -10.733874  Y17   Y18
## 1 AR18_Y17  -167.015338   -128.882101 AR18   Y17
```

Observations - Arc Lengths Densities & Summaries:* - **Robust Yield Relationships Across Time The arcs $Y17 \leftrightarrow Y18$ and $Y17 \leftrightarrow Y20$ consistently appear in both datasets, suggesting a temporal persistence of yield effects. This confirms that past yields are strong indicators of future performance — an important insight for agronomic forecasting and management planning.

- **Strong Seeding–Yield Dependency** The arc AR18 Y17 shows the strongest negative strength values in both raw and normalized datasets, indicating a strong inverse relationship between the seeding rate in 2018 and the yield in 2017. This may reflect over-seeding effects or resource competition, warranting further agronomic validation.
- **Normalization Preserves Direction and Signal** Although the magnitude of arc strength decreases post-normalization (e.g., from -167.0 to -128.9 in AR18 Y17), the direction and relative importance of relationships remain consistent. This supports the conclusion that normalization did not distort the data's structural dependencies — rather, it made them more comparable and stable, especially when combining data across years or scales.

- **Interpretability Enhanced** The consistent identification of these arcs in both forms of the dataset emphasizes the reliability of the learned structure, strengthening confidence in the model's ability to capture meaningful agronomic interactions.
- Only 4 common arcs between the two datasets.

F. FURTHER ANALYSIS of Variable Relationship

To further analysis whether a relationship exists between the yield of a given year on the yield or seeding of another or vice versa, (the seeding of a given year and its effect on the seeding or yield of another), we are proposing to use Linear Regression Model.

At the least, we expect $Y17$ to give strong insights into the yield for 2020 - based on the DAG plot. Particularly, we will examine the relationship among the variables for both the Raw and Normalized datasets to draw a conclusion.

1. Full Model Analyses: Raw and Normalized Data

```
# Raw Data Model - all variables
model_data_raw <- Combined3[, c("Y17", "AR18", "Y18", "Y19", "AR20", "Y20")]
model_data_raw <- na.omit(model_data_raw)

lm_raw <- lm(Y20 ~ Y17 + Y18 + Y19 + AR18 + AR20, data = model_data_raw)
cat("=== Linear Model: RAW Data - Y20 as Dependent Var ===\n")
```

```
## === Linear Model: RAW Data - Y20 as Dependent Var ===
```

```
print(summary(lm_raw))
```

```
##
## Call:
## lm(formula = Y20 ~ Y17 + Y18 + Y19 + AR18 + AR20, data = model_data_raw)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -72.878  -5.982   2.012   6.939  43.017
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.636e+02  7.922e+01   2.065 0.040207 *
## Y17          3.163e+00  5.905e-01   5.357 2.36e-07 ***
## Y18          2.338e-01  6.630e-02   3.526 0.000525 ***
## Y19          5.369e-01  4.572e-01   1.174 0.241734
## AR18        -9.192e-03  3.236e-03  -2.840 0.004982 **
## AR20         3.985e-04  3.501e-03   0.114 0.909485
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 15.36 on 196 degrees of freedom
## Multiple R-squared:  0.4781, Adjusted R-squared:  0.4648
## F-statistic: 35.91 on 5 and 196 DF,  p-value: < 2.2e-16
```

```r
# Normalized Data Model - all variables
colnames(wide_z5_95) <- sub("^z_5_95\\.", "z.", colnames(wide_z5_95))
model_data_z <- wide_z5_95[, c("z.Y17", "z.AR18", "z.Y18", "z.Y19", "z.AR20", "z.Y20")]
model_data_z <- na.omit(model_data_z)

lm_z <- lm(z.Y20 ~ z.Y17 + z.Y18 + z.Y19 + z.AR18 + z.AR20, data = model_data_z)
cat("=== Linear Model: Z-Score Data - Y20 as Dependent Var ===\n")
```

```
## === Linear Model: Z-Score Data - Y20 as Dependent Var ===
```

```r
print(summary(lm_z))
```

```
##
## Call:
## lm(formula = z.Y20 ~ z.Y17 + z.Y18 + z.Y19 + z.AR18 + z.AR20,
##     data = model_data_z)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.94958 -0.05210  0.03255  0.10371  0.37036
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.43929    0.88060   2.770  0.00614 **
## z.Y17        2.96972    0.72113   4.118 5.63e-05 ***
## z.Y18        0.17120    0.05931   2.886  0.00433 **
## z.Y19       -0.48041    0.60500  -0.794  0.42812
## z.AR18      -0.07988    0.04777  -1.672  0.09606 .
## z.AR20       0.11814    0.04556   2.593  0.01022 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1856 on 196 degrees of freedom
## Multiple R-squared:  0.4738, Adjusted R-squared:  0.4604
## F-statistic:  35.3 on 5 and 196 DF,  p-value: < 2.2e-16
```

1a. FULL MODEL ANALYSIS TABLES

```r
# Original wide data frame
tab <- data.frame(
  Raw = c("Significant Variables", "Intercept", "R-Squared", "P-Value", "Spec. Note"),
  Obs_1 = c("(Y17, Y18, AR18)", "163.6", "0.4781", "2.2e-16", "AR18 (-ve)"),
  Obs_2 = c("(Y17, Y18, AR20)", "2.4393", "0.4738", "2.2e-16", "AR20 (0.05 sig)"),
  stringsAsFactors = FALSE
)

# Transpose and clean up
tab_t <- as.data.frame(t(tab[ , -1]))  # transpose without first column
```

```r
# Assign new column names (from original first column)
colnames(tab_t) <- tab$Raw

# Add a new column with dataset labels
tab_t$Dataset <- c("Raw Data", "Z-Norm")

# Reorder to put Dataset first
tab_t <- tab_t[, c("Dataset", colnames(tab_t)[colnames(tab_t) != "Dataset"])]

# Print table with knitr::kable
library(knitr)
library(kableExtra)
```

Full Model Table

```
## Warning: package 'kableExtra' was built under R version 4.5.1
```

```r
kable(tab_t,
      caption = "Reduced Model Comparison Summary (Transposed)",
      align = "c") %>%
  kable_styling(position = "center", font_size = 9, latex_options = "hold_position") %>%
  add_footnote("Signif. codes: 0 '*** Very Strong' 0.001 '** Strong' 0.01 '* Moderate' 0.05 '. Weak' 0.
               notation = "symbol")
```

```
## Warning in add_footnote(., "Signif. codes: 0 '*** Very Strong' 0.001 '**
## Strong' 0.01 '* Moderate' 0.05 '. Weak' 0.1 ' ' 1", : Notation is set to
## 'number' and other formats are not supported.
```

Table 2: Reduced Model Comparison Summary (Transposed)

	Dataset	Significant Variables	Intercept	R-Squared	P-Value	Spec. Note
Obs_1	Raw Data	(Y17, Y18, AR18)	163.6	0.4781	2.2e-16	AR18 (-ve)
Obs_2	Z-Norm	(Y17, Y18, AR20)	2.4393	0.4738	2.2e-16	AR20 (0.05 sig)

INTERPRETATION - Full Model Analyses

Full Multiple Linear Models:

RAW DATASET EQUATION:

$$\hat{Y20} = 163.6 + 3.163 \cdot Y17 + 0.234 \cdot Y18 + 0.537 \cdot Y19 - 0.00919 \cdot AR_18 + 0.000398 \cdot AR_20$$

Z-SCALED DATASET EQUATION:

$$z.\hat{Y}20 = 2.439 + 2.970 \cdot z.Y17 + 0.171 \cdot z.Y18 - 0.480 \cdot z.Y19 - 0.0799 \cdot z.AR18 + 0.118 \cdot z.AR20$$

$Y20 \approx Y17 + Y18 + Y19 + AR18 + AR20$ and $z.Y20 \approx z.Y17 + z.Y18 + z.Y19 + z.AR18 + z.AR20$ for the given raw and normalized datasets respectively.

We note that: - Y17 and Y18 are strong predictors in both models with significance of at least 0.01 - an indication of high temporal dependency. However in the Raw dataset, Y18 has more significant power 0.001 while with the normalized dataset, z.Y18 has moderate significant power of 0.01.

- AR20 only proved significant in the normalized model at the 0.05 lower level.
- AR18 shows a slight negative spatial effect in both datasets
- Y19 and AR20 are useless as predictor variables and will not be further referenced in the next linear model phase.
- Adjusted R^2 for the raw data approx 0.4781 while in the normalized dataset approx 0.4738. In short, the un-norlmalized dataset has a slighly higher, though comparable R^2 value. Ajustted R^2 is the model's watchdog for non-contributing variables.
- The final model using untransformed (raw) predictors achieved an adjusted R^2 of 0.4648, slightly outperforming the normalized model (adjusted R^2= 0.4604). The significance of $Adjusted R^2$ is to adjust R^2 for the number of meaningless predictors in the model. it penalizes the model for adding variables that do not meaningfully improve the model.
- $z.Y17$ remains the most influential predictor
- $z.Y19$ becomes negatively associated - borderline case
- Normalization reveals hidden patterns, especially for $AR20$.
- Both models perform similarly, explaining $\approx$ 47%–48% of variation in $Y20$.

Independent variables in the raw data model explain 47% of variation, while 48% is explained in the normalized dataset. Against this backdrop, we may question the extra effort of normalization by a two-step process to produce about a 1% increased difference in explainability.

2. Reduced Model Analyses

```
# Clean raw data
model_data_raw <- Combined3[, c("Y17", "Y18", "Y20", "AR18")]
model_data_raw <- na.omit(model_data_raw)

# Build linear model with significant predictors
model_Y20_raw <- lm(Y20 ~ Y17 + Y18 + AR18, data = model_data_raw)

# Output summary
#summary(model_Y20_raw)
cat("=== Linear Model: RAW Data - Y20 as Dependent Var ===\n")
```

```
## === Linear Model: RAW Data - Y20 as Dependent Var ===
```

```
print(summary(model_Y20_raw))
```

```
##
## Call:
## lm(formula = Y20 ~ Y17 + Y18 + AR18, data = model_data_raw)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -77.556  -6.234   2.529   7.297  44.175
##
```

```
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) 166.298697  57.724972   2.881  0.00440 **
## Y17           3.350317   0.593191   5.648 5.58e-08 ***
## Y18           0.303885   0.060263   5.043 1.03e-06 ***
## AR18         -0.009029   0.003163  -2.855  0.00477 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 15.56 on 198 degrees of freedom
## Multiple R-squared:  0.4587, Adjusted R-squared:  0.4505
## F-statistic: 55.93 on 3 and 198 DF,  p-value: < 2.2e-16
```

```
# Ensure column names are standardized
colnames(wide_z5_95) <- sub("^z_5_95\\.", "z.", colnames(wide_z5_95))

# Clean normalized data
model_data_z <- wide_z5_95[, c("z.Y17", "z.Y18", "z.AR20", "z.Y20")]
model_data_z <- na.omit(model_data_z)

# Build linear model with significant predictors
model_zY20 <- lm(z.Y20 ~ z.Y17 + z.Y18 + z.AR20, data = model_data_z)

# Output summary
#summary(model_zY20)
cat("=== Linear Model: Z-Score Data - Y20 as Dependent Var ===\n")
```

```
## === Linear Model: Z-Score Data - Y20 as Dependent Var ===
```

```
print(summary(model_zY20))
```

```
##
## Call:
## lm(formula = z.Y20 ~ z.Y17 + z.Y18 + z.AR20, data = model_data_z)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.96845 -0.03497  0.02567  0.10214  0.38181
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.09255    0.36483   5.736 3.55e-08 ***
## z.Y17        1.93379    0.38073   5.079 8.66e-07 ***
## z.Y18        0.14290    0.05598   2.553 0.011431 *
## z.AR20       0.08185    0.02248   3.642 0.000345 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1871 on 200 degrees of freedom
## Multiple R-squared:  0.4641, Adjusted R-squared:  0.456
## F-statistic: 57.73 on 3 and 200 DF,  p-value: < 2.2e-16
```

2a. REDUCED MODEL ANALYSIS TABLES

```
# Original wide data frame
tab <- data.frame(
  Raw = c("Significant Variables", "Intercept", "R-Squared", "P-Value", "Spec. Note"),
  Obs_1 = c("(Y17, Y18, AR18)", "166.2987", "0.4587", "2.2e-16", "AR18 (-ve - 0.001)"),
  Obs_2 = c("(Y17, Y18, AR20)", "2.0926", "0.4641", "2.2e-16", "All pos coefs"),
  stringsAsFactors = FALSE
)

# Transpose and clean up
tab_t <- as.data.frame(t(tab[ , -1]))  # transpose without first column

# Assign new column names (from original first column)
colnames(tab_t) <- tab$Raw

# Add a new column with dataset labels
tab_t$Dataset <- c("Raw Data", "Z-Norm")

# Reorder to put Dataset first
tab_t <- tab_t[, c("Dataset", colnames(tab_t)[colnames(tab_t) != "Dataset"])]

# Print table with knitr::kable
library(knitr)
library(kableExtra)

kable(tab_t,
      caption = "Reduced Model Comparison Summary (Transposed)",
      align = "c") %>%
  kable_styling(position = "center", font_size = 9, latex_options = "hold_position") %>%
  add_footnote("Signif. codes: 0 '*** - Very Strong' 0.001 '** Strong' 0.01 '* Moderate' 0.05 '. Weak' 0
               notation = "symbol")
```

Reduced Model Table

```
## Warning in add_footnote(., "Signif. codes: 0 '*** - Very Strong' 0.001 '**
## Strong' 0.01 '* Moderate' 0.05 '. Weak' 0.1 ' ' 1", : Notation is set to
## 'number' and other formats are not supported.
```

Table 3: Reduced Model Comparison Summary (Transposed)

	Dataset	Significant Variables	Intercept	R-Squared	P-Value	Spec. Note
Obs_1	Raw Data	(Y17, Y18, AR18)	166.2987	0.4587	2.2e-16	AR18 (-ve - 0.001)
Obs_2	Z-Norm	(Y17, Y18, AR20)	2.0926	0.4641	2.2e-16	All pos coefs

ANALYSES OF REDUCED MODELs - Raw Dataset:

Resulting Equations:

$$\text{Y20} = 166.30 + 3.35 \cdot \text{Y17} + 0.30 \cdot \text{Y18} - 0.009 \cdot \text{AR18}$$

Notes:

- Maintained *Predictors*: (Y17, Y18, and AR18)
- $Y17$: typically positive (strong historical influence < 0.001) and $Y18$ is often positive, capturing immediate lag effect.
- $AR18$: Often negative possibly implying that seeding rate may penalize excessive or inefficient allocation.
- *Fit*: Moderate to strong R-squared depending on data quality.
- *Assumptions*: May suffer from multi-collinearity (Y17 and Y18 can be correlated).
- Potential heteroskedasticity (variance not constant).

ANALYSIS OF REDUCED MODEL - Normalized Dataset:

$$z(\text{Y20}) = 2.09 + 1.93 \cdot z(\text{Y17}) + 0.14 \cdot z(\text{Y18}) + 0.082 \cdot z(\text{AR20})$$

- Earlier years ($Y17, Y18$) have predictive value, but AR18 shows a suppressive (negative) impact, possibly indicating over-seeding or inefficiencies in 2018.
- Maintained *Predictors*: (z.Y17, z.Y18, and z.AR20)
- Standardization improves interpretability of effect size
- All predictors in this reduced model are on comparable scale
- *z.Y17* and *z.Y18* are positive
- z.AR20: May be negative (again implying diminishing returns on seeding)
- *Fit*: Similar R^2 value to raw model but potentially with more stable coefficients
- *Assumptions*: Normalization often helps meet linearity and homoscedasticity
- With a reduced variable model, it is easier to detect outliers or leverage points

Special Notes:

Moving to the *Reduced model*, we note that the predictive power of the retained $Y17$, $Y18$, and $AR18$ variables remained constant across all the models.

In contrast, in the *Reduced model*, only $zY17$ retained its strong significant strength of 0.001. zAR20's significant power increased dramatically from 0.05 (Full model) to 0.001 (Reduced model). A possible explanation for this observed behaviour: a statistical reflection of reduced competition for explanatory power, lower multicollinearity, and more focused modeling. This suggests that z.AR20 may be an important and reliable predictor of yield, with the removal of its confounding peers.

Additionally, $zY18's$ contribution slightly decreased from 0.01 (Full model) to 0.05(Reduced) model. The possible explanation: the slight decline in z.Y18's significance suggests that its predictive power may depend partly on its relationship with other variables. While still relevant, its role becomes more uncertain when viewed in isolation.

Even after normalization, seeding in 2020 (z.AR20) has limited or adverse influence, while prior yield continues to be a strong indicator of future yield, reaffirming the temporal inertia in the yield process.

CONCLUSION

1. Key Observations

Temporal Persistence of Yield:

** Prior yields (Y17, Y18) consistently exhibit strong positive effects on Y20, underscoring temporal persistence and the influence of environmental or management continuity.**

Seeding Rate Effects Variance: Seeding rates from different years (AR18 vs AR20) exhibit variable influences — sometimes negative - implying diminishing returns or potential inefficiencies in strategies.

Normalization Enhances Modeling Stability: Using normalized data (winsorized followed by z-score scaling) produces models with similar explanatory power but often more interpretable and stable coefficients, supporting better diagnostics.

Spatial Aggregation Matters: Aggregating data by spatial grid cells and filtering for sufficient observations is crucial to avoid noise and produce reliable, interpretable results.

Data Preparation is Critical: Careful spatial and temporal data alignment, combined with filtering and normalization, dramatically improves the quality and interpretability of models.

Model Parsimony Pays Off: Selecting only significant predictors simplifies interpretation without sacrificing predictive power which is important for actionable agronomic insights.

Causal Modeling ve Correlation: While both are welcomed and have their advantages, Causal Modeling can provide and re-inforce insight beyond Correlation. Bayesian network DAGs help clarify underlying dependencies and potential causal pathways, guiding more informed decision-making. Additionally, we extended this by analysing numerical strengths for the 2017-2020 acyclic plots.

Normalization is Relevant: In this context, normalization is recommended for comparative studies. It is essential for cross-year or treatment comparisons, ensuring consistent scale and preserving interpretability across plots.

Integrated Workflow is Essential: From raw data ingestion to final DAG visualization, maintaining clarity and rigor at each step is critical, especially when handling complex multi-year, multi-variable agricultural data.

The normalized and spatially aggregated data provided a structured foundation for modeling the relationship between harvest yield across years. Through Bayesian Network Analysis, we observed that prior yield and seeding rates influence current yield - consistent with the principles of crop rotation and management. Normalization effectively reduced the impact of outliers and improved comparability. We further conclude that results from Linear Regression (theoretic concept) supported those of the Bayesian MOdel (visual model).

2. Is there a relationship between the harvesting and/or seed application of prior years to a year's current yield?

From a visual perspective, we see that four arcs were consistently observed across both raw and normalized datasets, including strong dependencies such as AR18 Y17, Y17 Y18, and Y17 Y20. These reflect stable temporal and management-related relationships, confirming the importance of prior-years' yield and seeding rates in predicting current outcomes. Notably, the strength of arcs decreased under normalization, yet directionality and presence remained intact, highlighting that normalization improves comparability without distorting underlying agronomic structures.

The final equation we would use to explain the relationship between a year's yield and other predictors:

$$z(\mathbf{Y20}) = 2.09 + 1.93 \cdot z(\mathbf{Y17}) + 0.14 \cdot z(\mathbf{Y18}) + 0.082 \cdot z(\mathbf{AR20})$$

Even from our Multiple Regression Models show that the yields of at least two prior years' (e.g., Y17 and Y18) data significantly predict the yield of 2020, Y20.

Additionally, seeding rates (particularly AR18 and AR20) exert varied influence on yield, suggesting management effectiveness or diminishing returns. These relationships are consistent across both raw and normalized data and are supported by causal patterns found in Bayesian Network Analysis.

If our main goal were prediction accuracy alone, normalization may not be crucial here since R^2 remains constant. If, however, the goal is interpretablity, stable, and coparable coefficients across variables, then normalization is valuable.